

UNIVERSIDAD POPULAR DEL CESAR

Sistema de Gestión Hotelera — Hotel Nativo

Proyecto de Aula: Fases 1,2,3 y 4

Jarrison Pulgarín, Roberto Buelvas y Juan Buelvas

Facultad de Ingeniería y Tecnologías de la Información y la Comunicación, Universidad Popular del
Cesar

Programación de Computadores III (SS462)

Ing. Esp. Alfredo Bautista

11 de junio de 2026

Resumen

El presente documento presenta las Fases 1,2,3 y 4 del proyecto de aula Sistema de Gestión Hotelera Hotel Nativo, desarrollado para la asignatura Programación de Computadores III. La Fase 1 aborda la identificación, formulación y especificación del problema en el contexto de los hoteles medianos del departamento del Cesar, definiendo objetivos, requerimientos funcionales y no funcionales, y la estrategia de gestión del proyecto con Git. La Fase 2 presenta la arquitectura del sistema en tres capas, el diagrama de clases con 83 entidades del dominio hotelero, los mockups de las interfaces gráficas y la justificación de los principios de diseño orientado a objetos aplicados (patrones GRASP y principios SOLID). El sistema se desarrolla en Java con JavaFX y persistencia sobre Oracle Database, utilizando los patrones MVC y DAO. El resultado esperado es una aplicación de escritorio funcional, mantenible y alineada con buenas prácticas de ingeniería de software.

1. Introducción

1.1 Contextualización

Tras el pasar de los últimos años en Colombia, la industria del hotelería ha venido en un incremento que parece no tener fin, todo esto gracias a muchos colombianos que suelen viajar dentro del país, como también cada vez más extranjeros interesados por conocer nuestro país viajan a ciudades como lo es nuestra ciudad Valledupar. En este contexto el buen manejo de los recursos de un hotel es fundamental dichos recursos como lo son las habitaciones, el personal que trabaja en el hotel, las reservas y facturas del hotel tener un registro y control muy detallado de todo lo anterior se vuelve clave a la hora de tener una ventaja con la competencia, como también es importante para que los huéspedes que decidan quedarse se vayan con una buena experiencia.

sin embargo, en la actualidad la mayoría de hoteles de capacidad mediana en la región todavía tienen una dependencia a herramientas como lo son hojas de cálculos, cuadernos físicos para llevar registros o programas genéricos los cuales se les hace complicado su comprensión y buen manejo de estos programas esto genera un problema: se puede llegar a perder información ya sea por fallos humanos o algún otro factor que pueda dañar los registros físicos ocasionando que puedan llegar a confundirse que habitación tienen o no tienen disponible , aun más grave esto puede ocasionar que las facturas queden incoherentes y cueste mucho el análisis de cómo va el negocio. Al no contar con un sistema integrado que lleve el conteo, el manejo o control de estos datos se detiene el crecimiento del hotel y esto visto desde la parte administrativa se hace complejo el tomar decisiones referentes al negocio a tiempo.

1.2 Importancia

Importancia del proyecto

Este proyecto no tiene el fin de ser solo un ejercicio académico sin mas. Tiene un valor firme real tanto para el grupo de desarrolladores como para ámbito laboral que busca ser de utilidad.

para el equipo de desarrolladores, elaborar un sistema de gestión hotelera en java, con una arquitectura dividida en capas, persistencia relacional y un diseño sólidamente basado en la programación orientada a objetos, esto representa un gran desafío formativo pues aquí se pone en practica las competencias esenciales como son:

saber modelar correctamente un dominio real, saber aplicar bien herencia como también polimorfismo donde tengan sentido, implementar el patrón DAO para las responsabilidades, diseñar las interfaces graficas funcionales con JavaFX y sobre todo escribir el código de manera que otro programador lo pueda entender para facilitar las tareas de mantenimiento. No se trata de hacer el proyecto que funcione por funcionar si no hacerlo bien completo.

Desde un punto de vista del negocio, la situación se hace clara: la mayoría de hoteles medianos en la región aun manejan herramientas que ocasionan errores desorden y perdidas de tiempo valioso. Una solución como hotel Nativo llega para resolver todos estos errores que pueden llegar a ocasionar en casos extremos perdidas monetarias. Ofreciendo un seguimiento completo del huésped desde el momento que hace la reserva hasta el check-out, controlando en tiempo real la disponibilidad de las habitaciones, facturaciones automatizadas sin incoherencias y reportes históricos que le permitan al personal administrativo tomar decisiones con una base de información de alta confianza.

1.3 Breve descripción de la solución

Se va a desarrollar una aplicación de escritorio, pensada para usarse día a día en la recepción y la administración de un hotel de tamaño real: cuatro pisos, veintiocho habitaciones, y cuatro tipos de habitaciones que responden a distintas necesidades de los huéspedes: Simples (para quienes viajan solos), Múltiples (ideales para grupos o familias), Dobles (la clásica para parejas o colegas) y Suites (para quienes buscan más espacio y comodidad).

El sistema no será un simple buscador de disponibilidad. Tendrá módulos bien definidos que cubren todo lo que pasa en un hotel:

- **Gestión de clientes:** para tener los datos de quienes se hospedan.
- **Gestión de habitaciones:** para saber en todo momento qué está ocupado, libre o en mantenimiento.
- **Reservas:** para apartar con anticipación sin confusiones.
- **Check-in y check-out:** el corazón de la operación diaria.
- **Facturación con IVA:** porque las cuentas claras evitan problemas.
- **Mantenimiento de infraestructura:** para que una habitación con una lámpara rota no se asigne por error.
- **Reportes:** para que el administrador sepa qué pasó, qué se facturó y cómo va el negocio.
- **Autenticación de empleados:** porque no cualquiera debe tener acceso a todo.

Todo esto se guardará en una base de datos Oracle, usando el patrón DAO para que el código sea ordenado y fácil de mantener. La interfaz se hará con JavaFX 21 sobre Java 17, siguiendo el

patrón MVC: los archivos FXML se encargan de la pantalla, y el controlador de la lógica, separando bien las cosas para que el proyecto no se vuelva un enredo.

2. Planteamiento del problema

2.1 Identificación del problema

Los hoteles de tamaño mediano en ciudades como Valledupar, Sincelejo o Riohacha están creciendo, pero muchos siguen manejando sus operaciones con herramientas que ya no les dan abasto. No tienen un sistema integrado, y eso se traduce en problemas del día a día que duelen en el bolsillo y en la reputación del negocio. Estos son los más graves:

- **Doble asignación de habitaciones.** Pasa cuando no hay forma de verificar en tiempo real si una habitación está libre o ya la reservó otro huésped. El resultado: clientes molestos y recepcionistas estresados.
- **Se pierde el historial de los huéspedes.** Sin una base de datos centralizada, los datos de quienes se hospedaron quedan en papeles sueltos o archivos que nadie vuelve a encontrar. Así no se puede fidelizar ni conocer al cliente.
- **Errores en las facturas.** Al calcular tarifas, descuentos e IVA a mano o con hojas de cálculo improvisadas, los errores son casi inevitables. A veces se cobra de menos, a veces de más, y ninguna de las dos opciones es buena.

- **Mantenimiento olvidado.** No hay forma de programar ni dejar registro de cuándo se reparó una habitación. Entonces una ducha roja o un aire acondicionado dañado terminan asignándose por error a un huésped. Mala experiencia garantizada.
- **No hay reportes confiables.** El gerente no tiene datos claros para saber cuánto se vendió, qué habitaciones rinden más o cómo va la ocupación. Toma decisiones a ojo, y eso es un lujo que ningún negocio puede darse.
- **Todo depende de procesos manuales.** Y donde hay manual, hay error humano: anotaciones mal leídas, cuentas mal sumadas, papeles que se mojan o se pierden. La información no está segura ni disponible cuando se necesita.

En pocas palabras: el hotel funciona, pero funciona mal, con costos invisibles que van subiendo y oportunidades que se escapan. Ese es el problema que este proyecto busca resolver.

2.2 Justificación

La construcción del Sistema de Gestión Hotelera Hotel Nativo se justifica desde tres dimensiones complementarias:

Dimensión académica: aprender haciendo, pero haciendo algo real

Este proyecto es el escenario perfecto para poner en práctica todo lo visto en Programación de Computadores III. No se trata de ejercicios de laboratorio aislados, sino de un problema real que obliga a usar herencia, polimorfismo, interfaces, colecciones, manejo de excepciones, conexión a bases de datos y construcción de interfaces gráficas todo al mismo tiempo.

Además, la complejidad de un hotel —con sus tipos de habitaciones, reservas, facturación y roles de empleados— fuerza al equipo a aplicar buenos principios de diseño como SOLID y **GRASP**. El objetivo no es que el código "funcione a duras penas", sino que esté bien organizado, con bajo acoplamiento, fácil de extender y sin miedo a meterle mano más adelante. Eso es lo que separa un proyecto escolar de un proyecto profesional.

Dimensión tecnológica: herramientas que sí se usan en el mundo real

No estamos inventando tecnologías raras ni aprendiendo cosas que quedarán obsoletas al otro año. Java con JavaFX es una combinación madura, multiplataforma y con décadas de uso en aplicaciones empresariales de escritorio. No es un juguete, es una herramienta de verdad.

Y del lado de la base de datos, Oracle Database no fue elegido por moda: ofrece robustez, transacciones confiables y un modelo relacional que garantiza que los datos del hotel —habitaciones, huéspedes, facturas— no se corrompan ni se pierdan. Trabajar con este stack expone al estudiante a un entorno profesional vigente, no a un simulacro.

Dimensión social y económica: tecnología que puede ayudar a hoteles de verdad

Aquí está el corazón del asunto. Un sistema como Hotel Nativo no tiene por qué quedarse en un repositorio universitario. Puede ser adoptado por hoteles reales del Cesar y de toda la región Caribe que hoy operan con cuadernos y hojas de cálculo. Esa modernización no es un lujo: es una necesidad para mejorar el servicio, reducir errores y ser más competitivos.

Y Valledupar no es cualquier ciudad. Es la sede del Festival de la Leyenda Vallenata, un destino turístico que crece cada año. Cada vez llegan más viajeros nacionales e internacionales, y los hoteles independientes necesitan herramientas tecnológicas que puedan pagar y usar sin complicaciones. Una solución como esta puede ayudar a que pequeños hoteleros compitan en mejores condiciones, lo que a su vez dinamiza el turismo local y sostiene empleos.

En resumen: hacemos este proyecto porque aprenderemos mucho, usaremos tecnologías de verdad y, con suerte, construiremos algo que alguien pueda aprovechar para mejorar su negocio. No está nada mal.

2.3 Estado del arte

A nivel mundial, los sistemas de gestión hotelera (PMS) más conocidos son Opera Cloud, Cloudbeds, Mews y Little Hotelier. Son soluciones potentes, funcionan en la nube y ofrecen desde reservas hasta análisis de datos. Pero tienen un problema: están pensados para grandes cadenas hoteleras. Sus costos de licencia y requisitos técnicos los hacen inaccesibles para un hotel mediano e independiente en Valledupar.

En el contexto académico colombiano, universidades como la Nacional, Los Andes y Antioquia han trabajado sistemas de reservas como casos de estudio, usando tecnologías como Spring Boot, Hibernate y MySQL. Son proyectos valiosos, pero suelen enfocarse en arquitectura web y frameworks que ocultan gran parte del trabajo con la base de datos.

Hotel Nativo se diferencia porque apuesta por JavaFX con FXML para construir una interfaz de escritorio clásica pero moderna. Usa Oracle XE como base de datos y, sobre todo, implementa el patrón DAO de forma explícita, sin frameworks de persistencia. ¿Por qué? Para que el estudiante comprenda desde sus cimientos cómo se mapea un objeto a una tabla relacional.

En cuanto a patrones de diseño, el proyecto aplica los más reconocidos para el dominio hotelero: Strategy para descuentos dinámicos (VIP, temporada baja), DAO para acceso a datos, MVC para separar la lógica de la interfaz y Singleton para manejar la conexión a la base de datos. Cada uno resuelve un problema real del día a día en un hotel.

3. Objetivos

3.1 Objetivo general

Diseñar e implementar un Sistema de Gestión Hotelera desarrollado en Java, con interfaz gráfica en JavaFX y persistencia en base de datos Oracle, que permita automatizar los procesos clave del Hotel Nativo: administración de habitaciones, reservas, check-in y check-out, facturación y mantenimiento.

3.2 Objetivos específicos

1. **Modelar el problema del hotel** a través de un diagrama de clases que use herencia, interfaces y clases abstractas. Allí deben quedar representadas las entidades principales como Persona, Cliente, Empleado, Habitación, Reserva, Check-In-Out, Factura y Mantenimiento, con sus atributos, métodos y relaciones bien definidas.
2. **Construir la capa de acceso a datos** aplicando el patrón DAO, con interfaces y clases concretas para cada entidad. Esto permitirá hacer operaciones CRUD (crear, leer, actualizar, eliminar) sobre una base de datos Oracle XE, manejando las excepciones de forma adecuada para que el sistema no se caiga ante cualquier error inesperado.
3. **Desarrollar la lógica de negocio** en una capa de servicios independiente. Allí se aplicarán reglas de validación, se calcularán tarifas con descuentos diferenciados (por ejemplo, para clientes VIP o en temporada baja) y se gestionarán los estados de reservas y habitaciones (disponible, ocupada, en mantenimiento, etc.).
4. **Crear la interfaz gráfica** con JavaFX y FXML, siguiendo el patrón MVC para mantener limpia la separación entre la pantalla y la lógica. Se construirán vistas funcionales para cada módulo del

sistema: Login, Dashboard, Clientes, Habitaciones, Reservas, Check-in/Check-out, Facturación, Mantenimiento y Reportes.

5. **Gestionar el proyecto con Git** usando una estrategia de ramas por integrante, de manera que cada quien trabaje en la suya y luego se integre a la rama principal main. Así se podrá llevar un historial claro de contribuciones individuales y asegurar que la integración continua no genere conflictos ni pérdida de trabajo.

El Sistema de Gestión Hotelera Hotel Nativo es una aplicación de escritorio desarrollada en Java 17 con interfaz gráfica JavaFX 21 y persistencia sobre Oracle Database XE 21c. La solución implementa una arquitectura en tres capas (presentación, lógica de negocio y acceso a datos) bajo los patrones MVC y DAO, con módulos para autenticación, gestión de clientes y empleados, administración de habitaciones, reservas, check-in/check-out, facturación con IVA, mantenimiento y reportes operativos. Los descuentos para clientes VIP y temporada baja se modelan con el patrón Strategy y la conexión a base de datos se gestiona mediante Singleton. La aplicación está pensada para hoteles medianos con cuatro pisos y veintiocho habitaciones distribuidas en tipos Simple, Múltiple, Doble y Suite.

4. Descripción de la solución

4.1 Descripción general

4.2 Requerimientos funcionales

RF01 – Autenticación de Empleados

- El sistema debe permitir que los empleados registrados inicien sesión usando su correo electrónico y contraseña.
- Debe distinguir al menos dos perfiles: Administrador (puede hacer todo) y Recepcionista (solo lo operativo).
- Si las credenciales son incorrectas, debe bloquear el acceso y mostrar un mensaje de error claro.

RF02 – Gestión de Clientes

- Se debe poder registrar clientes con: nombre, apellido, correo, teléfono, número de documento, tipo de documento (colombiana o extranjera), nacionalidad y ciudad de origen.
- Debe permitir consultar, editar y buscar clientes por número de documento o por nombre.
- Debe identificar automáticamente a los clientes VIP y aplicarles descuentos especiales en sus reservas.

RF03 – Gestión de Habitaciones

- El hotel tiene 28 habitaciones distribuidas en cuatro pisos, con los siguientes tipos y tarifas por noche:
 - Simple: \$80.000
 - Múltiple de 2 camas: \$120.000
 - Múltiple de 3 camas: \$180.000
 - Doble: \$100.000
 - Suite: \$150.000

- Cada habitación debe tener un estado: Disponible, Ocupada, En Mantenimiento o Fuera de Servicio.
- Debe ser posible buscar habitaciones disponibles por rango de fechas, tipo y cantidad de personas.

RF04 – Gestión de Reservas

- Se debe poder crear una reserva asignándole: un cliente, una habitación, fechas de entrada y salida, y número de personas. El sistema debe validar disponibilidad y capacidad de la habitación.
- Los estados de una reserva siguen este flujo: Pendiente - Confirmada - En Proceso - Completada. En cualquier etapa activa también se puede Cancelar.
- El sistema debe calcular automáticamente el precio total de la reserva, aplicando descuentos por VIP o por temporada baja si corresponde.
- Al confirmar o cancelar una reserva, debe enviarse una notificación por correo electrónico al cliente.

RF05 – Gestión de Check-in / Check-out

- Se debe poder hacer el check-in de una reserva que esté en estado Confirmada. Al hacerlo, la habitación pasa a Ocupada y la reserva avanza a En Proceso.
- Al hacer el check-out, la habitación queda disponible nuevamente y la reserva se marca como Completada.
- El sistema debe hacer un check-out automático a las 11:00 AM para aquellas reservas cuya fecha de salida ya haya vencido.

RF06 – Facturación

- Al completar el check-out de una reserva, el sistema debe generar automáticamente una factura que incluya: subtotal, IVA (19%) y total.
- Se debe registrar el método de pago: efectivo, tarjeta de crédito, tarjeta de débito o transferencia.
- La factura se debe poder generar en formato HTML para imprimirla o enviarla por correo electrónico.

RF07 – Gestión de Mantenimiento

- Se debe poder registrar solicitudes de mantenimiento para habitaciones, clasificándolas en: Preventivo, Correctivo o Emergencia.
- Cada solicitud debe tener un empleado técnico responsable, fecha de solicitud y fecha de realización.
- Los estados del mantenimiento son: Solicitado, En Proceso, Completado y Cancelado.

RF08 – Reportes

- El sistema debe generar los siguientes reportes:
 - Ocupación hotelera por período
 - Listado de reservas activas y completadas
 - Historial de facturas
 - Resumen de mantenimientos
- Todos los reportes deben poder verse en pantalla o exportarse en formato HTML.

RF09 – Gestión de Empleados

- El Administrador debe poder registrar, consultar y editar empleados, incluyendo su cargo, salario base y datos de contacto.
- Los cargos que debe soportar el sistema son: Recepcionista, Aseadora, Administrador, Contador, Vigilante, Técnico de Mantenimiento y Técnico de Sistemas.

4.3 Requerimientos no funcionales

RNF01 – Rendimiento

- Las consultas a la base de datos no deben demorar más de 2 segundos cuando se estén leyendo tablas con menos de 10.000 registros.
- El sistema debe aguantar que al menos dos empleados trabajen al mismo tiempo sin que se note lentitud o se traben.

RNF02 – Usabilidad

- La interfaz gráfica debe ser fácil de entender y usar, con un menú lateral o un panel principal que permita moverse entre módulos sin perderse.
- Los formularios deben validar los datos mientras el usuario los escribe, mostrando mensajes de error claros y en español para que se entienda qué salió mal.
- La aplicación debe arrancar en menos de 5 segundos en los computadores normales del laboratorio de la universidad.

RNF03 – Mantenibilidad

- El código debe seguir las convenciones de nombres de Java: camelCase para métodos y atributos, PascalCase para clases, y las buenas prácticas del libro *Código Limpio* de Robert C. Martin.
- Cada clase pública debe tener un comentario Javadoc explicando qué hace, cuáles son sus atributos importantes y qué métodos ofrece.
- La arquitectura en capas debe asegurar que si se cambia algo en la pantalla, no se rompa la lógica del negocio ni el acceso a la base de datos.

RNF04 – Portabilidad

- La aplicación debe poder correr en Windows 10 u 11, siempre que tenga instalado Java 17 y JavaFX 21.
- El script que crea las tablas de la base de datos debe funcionar en Oracle XE 21c sin modificaciones.

RNF05 – Seguridad

- En esta versión académica, las contraseñas de los empleados se guardarán en texto plano en la base de datos, pero se dejará claro en la documentación que en un sistema real deberían encriptarse con algo como BCrypt.
- Solo los usuarios con rol Administrador pueden entrar a los módulos de administración; esto se debe validar en la capa de servicios, no solo en la pantalla.

RNF06 – Disponibilidad y Confiabilidad

- Si se pierde la conexión a la base de datos, el sistema no debe colapsar. Debe mostrar un mensaje amigable al usuario y registrar el error en la consola, pero seguir funcionando cuando se restablezca la conexión.

5. Gestión del proyecto con Git

El proyecto se desarrolla siguiendo una estrategia de control de versiones con Git que garantiza trazabilidad, trabajo paralelo entre los integrantes del equipo y la posibilidad de revertir cambios cuando sea necesario. A continuación se describen el repositorio, los roles, la estrategia de ramas y la distribución de tareas.

5.1 Repositorio

El código fuente se aloja en un repositorio privado en GitHub denominado hotel-system-nativo. La rama principal main contiene siempre la versión estable del sistema; las funcionalidades nuevas se desarrollan en ramas independientes y se integran mediante pull requests revisados por al menos un integrante del equipo.

5.2 Roles del equipo

Jarrison Pulgarín actúa como líder del proyecto, encargado de la coordinación general, revisión de pull requests y aseguramiento de la calidad del código. Roberto Buelvas asume el rol de desarrollador principal en la capa de persistencia y modelado de datos. Juan Buelvas se encarga del diseño de la interfaz gráfica y de la integración con la capa de servicios.

5.3 Estrategia de ramas

El equipo trabaja con un esquema de una rama por integrante derivada de main: feature/jarrison, feature/roberto y feature/juan. Cada integrante desarrolla sus tareas en su rama y las integra a main mediante pull requests. Esta estrategia evita conflictos directos sobre la rama principal y mantiene un historial limpio.

5.4 Distribución de tareas

Las tareas se asignan según los roles definidos: Jarrison desarrolla los servicios de Reserva y Habitación y la lógica de negocio principal; Roberto implementa las clases del modelo, los DAO y la conexión con la base de datos Oracle; Juan diseña los formularios JavaFX y conecta la interfaz con los servicios. Cada integrante documenta sus commits con mensajes descriptivos.

6. Conclusiones

Hasta este punto del proyecto —identificación, formulación y especificación del problema— hemos logrado delimitar con claridad qué debe hacer el Sistema de Gestión Hotelera Hotel Nativo.

Definimos nueve módulos funcionales esenciales y seis categorías de requerimientos no funcionales que nos servirán como guía en las fases que vienen.

Al contextualizar el problema, confirmamos algo importante: en los hoteles de ciudades intermedias colombianas hay una necesidad real de tecnología accesible. Esto le da al proyecto una relevancia que va más allá del aula: tiene un valor académico, sí, pero también social.

El análisis del estado del arte nos mostró que las soluciones comerciales que existen hoy son caras y están pensadas para cadenas grandes. Eso valida nuestro enfoque: construir una alternativa viable, educativa y práctica. La decisión de usar Java con JavaFX, Oracle XE y el patrón DAO no fue casual. Buscamos una plataforma madura que permita aprender a fondo el mapeo objeto-relacional y que, de paso, exponga al equipo a un stack profesional real.

La definición detallada de los requerimientos funcionales y no funcionales se convierte en la base contractual del proyecto. Con esto, el equipo puede planificar el trabajo por fases, sabiendo exactamente qué se espera en cada entrega.

En las fases siguientes, nos enfocaremos en el diseño de la arquitectura, la construcción del diagrama de clases y la implementación progresiva de los módulos. Siempre teniendo como hilo conductor el ciclo del huésped: desde que reserva hasta que hace check-out y recibe su factura. Ese es el corazón del sistema.

7. Referencias

Martin, R. C. (2008). Clean Code: A Handbook of Agile Software Craftsmanship. Prentice Hall.

Larman, C. (2004). Applying UML and Patterns: An Introduction to Object-Oriented Analysis and Design and Iterative Development (3rd ed.). Prentice Hall.

Martin, R. C. (2000). Design Principles and Design Patterns. Object Mentor. Recuperado de https://staff.cs.utu.fi/~jounsmed/doos_06/material/DesignPrinciplesAndPatterns.pdf

Oracle Corporation. (2023). Oracle Database XE 21c Documentation. Recuperado de <https://docs.oracle.com/en/database/oracle/oracle-database/21/xeinl/index.html>

OpenJFX. (2023). JavaFX 21 Documentation. Recuperado de <https://openjfx.io/javadoc/21/>

COTELCO – Asociación Hotelera y Turística de Colombia. (2023). Informe de indicadores hoteleros Colombia 2023. Bogotá: COTELCO.

Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). Design Patterns: Elements of Reusable Object-Oriented Software. Addison-Wesley.

Pressman, R. S., & Maxim, B. R. (2020). Ingeniería del Software: Un enfoque práctico (8.a ed.). McGraw-Hill

1. Definición de la Arquitectura

El sistema Hotel Nativo está diseñado bajo una arquitectura en tres capas (3-Tier Architecture), la cual separa las responsabilidades del sistema en niveles bien definidos: Presentación, Lógica de Negocio y Acceso a Datos. Esta separación garantiza bajo acoplamiento, alta cohesión y facilita el mantenimiento y escalabilidad del proyecto.

1.1 Diagrama de Paquetes Organizados por Capas

Capa	Paquetes Java	Responsabilidad
Presentación	com.hotel.ui.controllers com.hotel.ui.fxml com.hotel.ui.styles com.hotel.ui.components com.hotel.view	Controladores FXML, archivos de interfaz gráfica (.fxml), estilos CSS, utilidades de navegación y vistas de consola.
Lógica de Negocio	com.hotel.service com.hotel.model com.hotel.dto com.hotel.exception	Servicios con reglas de negocio, modelos del dominio, objetos de transferencia de datos (DTO) y excepciones personalizadas.
Acceso a Datos	com.hotel.dao.interfaces com.hotel.dao.impl com.hotel.util	Interfaces DAO, implementaciones JDBC con Oracle, utilidades de conexión y validación de entradas.
Aplicación	com.hotel	Punto de entrada (HotelApp), contexto de aplicación (ApplicationContext) y definición del módulo Java (module-info.java).

2. Diagrama de Clases

2.1 Identificación de Clases — Tabla Completa (106 clases)

La siguiente tabla identifica todas las clases del sistema con su tipo y rol dentro de la arquitectura:

#	Nombre de la Clase	Tipo	Descripción y Rol en el Sistema
---	--------------------	------	---------------------------------

1	HotelApp	Concreta	Punto de entrada de la aplicación JavaFX. Extiende Application, carga Login.fxml y configura el Stage principal.
2	AppContext	Concreta	Contexto de la aplicación. Instancia y provee todos los servicios y DAOs en un único punto de composición (Contenedor IoC manual).
3	Persona	Abstracta	Clase base que centraliza atributos de identidad personal (id, nombre, apellido, email, teléfono) comunes a Cliente y Empleado.
4	Cliente	Concreta	Extiende Persona. Representa al huésped del hotel con documento, nacionalidad, subtipo de documento y estado VIP.
5	Empleado	Concreta	Extiende Persona. Representa al personal del hotel con cargo asignado y fecha de contratación.
6	Habitacion	Concreta	Entidad física del hotel. Gestiona su propio estado (DISPONIBLE, OCUPADA, MANTENIMIENTO, FUERA_DE_SERVICIO). Aplica GRASP Experto.
7	TipoHabitacion	Abstracta	Jerarquía de tipos de habitación. Define calcularPrecio() y obtenerEtiquetaTipo() como métodos abstractos. Aplica SOLID OCP.
8	HabitacionSimple	Concreta	Extiende TipoHabitacion. Habitación para 1 persona, multiplicador de precio 1.0. Retorna etiqueta SIMPLE.
9	HabitacionDoble	Concreta	Extiende TipoHabitacion. Habitación para 2 personas, multiplicador 1.0. Precio base almacenado directamente en BD.
10	HabitacionMultiple	Concreta	Extiende TipoHabitacion. Para 2 o 3 camas (120.000 / 180.000 COP). Capacidad máxima 4 personas.

11	HabitacionSuite	Concreta	Extiende TipoHabitacion. Suite de lujo, capacidad 4 personas, amenities premium. Multiplicador 1.0 con precio base premium.
12	Reserva	Concreta	Asocia Cliente con Habitacion en un rango de fechas. Gestiona ciclo de vida: PENDIENTE → CONFIRMADA → EN_PROCESO → COMPLETADA.
13	Factura	Concreta	Documento de cobro. Calcula subtotal, IVA (19%) y total. Maneja estados PENDIENTE, PAGADA, ANULADA y métodos de pago.
14	CheckInOut	Concreta	Registra el ingreso y salida efectiva del huésped con fecha/hora real, empleado responsable y observaciones.
15	Mantenimiento	Concreta	Orden de mantenimiento sobre una habitación. Tipos: PREVENTIVO, CORRECTIVO, EMERGENCIA. Estados: SOLICITADO, EN_PROCESO, COMPLETADO.
16	Cargo	Concreta	Cargo laboral del empleado. Tiene nombre, descripción y salario base. Usado por Empleado para definir su rol.
17	Piso	Concreta	Piso físico del hotel con número, descripción y cantidad de habitaciones. Referenciado por Habitacion.
18	Servicio	Concreta	Servicio adicional del hotel (spa, restaurante, etc.) con precio y disponibilidad. Pendiente de integración completa.
19	ReservaService	Concreta	Controlador GRASP que orquesta la creación, confirmación y cancelación de reservas. Aplica SOLID SRP y DIP.
20	ClienteService	Concreta	Lógica de negocio para registro, actualización y búsqueda de clientes. Aplica GRASP Alta Cohesión.
21	HabitacionService	Concreta	Lógica de negocio para gestión de habitaciones. Maneja estados y delega a TipoHabitacion el cálculo de precios.

22	EmpleadoService	Concreta	Lógica de negocio para gestión de empleados. Integra con AuthService para registrar credenciales al crear empleados.
23	CheckInOutService	Concreta	Coordina el flujo de check-in y check-out. Transiciona habitaciones y reservas. Incluye checkouts automáticos a las 11:00 AM.
24	FacturaService	Concreta	Genera y gestiona facturas a partir de reservas completadas. Aplica IVA y método de pago.
25	MantenimientoService	Concreta	Registra órdenes de mantenimiento, bloquea habitaciones automáticamente y notifica técnicos via EmailService.
26	ReporteService	Concreta	Genera reportes de ocupación e ingresos en formato HTML. Filtra reservas directamente en BD via IReservaBusqueda.
27	AuthService	Concreta	Gestiona autenticación, sesiones con tokens, roles (ADMINISTRADOR, RECEPCIONISTA, MANTENIMIENTO) y expiración (8 horas).
28	ChatbotService	Concreta	Chatbot con NLP básico. Detecta intenciones (disponibilidad, estado reserva, precios) y responde con datos reales de BD.
29	EmailService	Concreta	Envía notificaciones por correo a clientes (confirmación reserva, cancelación, factura) y a empleados (mantenimiento).
30	Descuento	Interfaz	Contrato para el patrón Strategy de descuentos. Define calcularDescuento() y obtenerDescripcion(). Aplica SOLID OCP.
31	DescuentoVip	Concreta	Implementa Descuento. Aplica 15% de descuento a clientes VIP usando la constante de Constantes.DESCUENTO_CLIENTE_VIP.
32	DescuentoTemporadaBaja	Concreta	Implementa Descuento. Aplica 10% de descuento en temporada baja usando Constantes.DESCUENTO_TEMPORADA_BAJA.

33	SinDescuento	Concreta	Implementa Descuento con monto 0.0. Patrón Null Object, evita verificaciones null en ReservaService.
34	IReservaDAO	Interfaz	Contrato CRUD para Reserva: insertar, actualizar, eliminar, buscarPorId, listarTodas. Aplica SOLID ISP y DIP.
35	IReservaBusqueda	Interfaz	Búsquedas avanzadas de reservas: por cliente, solapadas, por rango de fechas. Segregada del CRUD básico.
36	IClienteDAO	Interfaz	Contrato CRUD para Cliente. Separa operaciones básicas de búsquedas avanzadas (IClienteBusqueda).
37	IClienteBusqueda	Interfaz	Búsquedas avanzadas de clientes: por documento, nombre, email y clientes VIP. Aplica SOLID ISP.
38	IHabitacionDAO	Interfaz	Contrato para Habitacion: CRUD, buscarDisponibles() con criterios complejos y actualizarEstado(). Aplica GRASP Bajo Acoplamiento.
39	IEmpleadoDAO	Interfaz	Contrato para Empleado: CRUD y búsqueda por cargo. Usado por EmpleadoService y AuthService.
40	IFacturaDAO	Interfaz	Contrato para Factura: insertar, actualizar, buscar por reserva, listar por cliente. Aplica SOLID DIP.
41	ICheckInOutDAO	Interfaz	Contrato para CheckInOut: CRUD y buscarCheckinsActivosPendientesCheckout() para checkout automático diario.
42	IMantenimientoDAO	Interfaz	Contrato para Mantenimiento: CRUD, listarPorHabitacion() y listarPendientes(). Aplica GRASP Indirección.
43	ReservaDAOImpl	Concreta	Implementación JDBC de IReservaDAO e IReservaBusqueda. Ejecuta queries SQL contra Oracle para todas las operaciones de reserva.

44	ClienteDAOImpl	Concreta	Implementación JDBC de IClienteDAO e IClienteBusqueda. Mapea ResultSet a objetos Cliente. Extiende BaseDAO.
45	HabitacionDAOImpl	Concreta	Implementación JDBC de IHabitacionDAO. Incluye query de disponibilidad con JOIN contra reservas activas.
46	EmpleadoDAOImpl	Concreta	Implementación JDBC de IEmpleadoDAO. Mapea empleados con su cargo desde Oracle. Extiende BaseDAO.
47	FacturaDAOImpl	Concreta	Implementación JDBC de IFacturaDAO. Gestiona persistencia de facturas con cálculo de IVA almacenado.
48	CheckInOutDAOImpl	Concreta	Implementación JDBC de ICheckInOutDAO. Incluye consulta de check-ins activos pendientes de checkout automático.
49	MantenimientoDAOImpl	Concreta	Implementación JDBC de IMantenimientoDAO. Gestiona órdenes de mantenimiento con sus estados y empleado asignado.
50	BaseDAO	Abstracta	Clase base para todos los DAOs. Provee acceso al pool de conexiones y manejo centralizado de excepciones SQL.
51	ReservaDTO	Concreta	DTO para crear una reserva. Transporta idCliente, idHabitacion, fechaEntrada, fechaSalida y numPersonas.
52	ClienteDTO	Concreta	DTO para crear/actualizar un cliente. Incluye subtipoDocumento y ciudadOrigen además de datos básicos de Persona.
53	HabitacionDTO	Concreta	DTO para crear/actualizar habitación. Transporta numero, idTipoHabitacion, idPiso y precioBase.
54	BusquedaDisponibilidadDTO	Concreta	DTO que encapsula criterios de búsqueda de disponibilidad: fechas, tipo de habitación, personas y precio máximo.

55	ExcepcionNegocio	Concreta	Excepción base para errores de lógica de negocio. Incluye codigoError para identificar el tipo de falla. Aplica SOLID SRP.
56	ExcepcionValidacion	Concreta	Extiende BusinessException. Específica para errores de validación, incluye el nombre del campo inválido.
57	ExcepcionBaseDatos	Concreta	RuntimeException para errores de infraestructura de base de datos. Separa errores de BD de errores de negocio.
58	HabitacionNoDisponibleExcepcion	Concreta	Extiende BusinessException. Lanzada cuando se intenta reservar una habitación no disponible para las fechas.
59	ExcepcionReservaNoEncontrada	Concreta	Extiende BusinessException. Lanzada cuando no se encuentra una reserva por su ID.
60	ConexionBaseDeDato	Concreta	Singleton con pool de 5 conexiones JDBC a Oracle. Inicialización LAZY para no bloquear el hilo de JavaFX al arrancar.
61	Constantes	Concreta	Repositorio de constantes del dominio: TASA_IVA (19%), descuentos, límites de check-in/out y tamaño del pool de BD.
62	ValidadorEntradas	Concreta	GRASP Fabricación Pura. Centraliza validaciones reutilizables: email, teléfono, IDs, fechas, campos requeridos.
63	LoginController	Concreta	Controlador FXML de la pantalla de Login. Gestiona autenticación, muestra errores y navega al Dashboard.
64	DashboardController	Concreta	Controlador FXML del Dashboard. Muestra métricas de ocupación, reservas del día y accesos a módulos.
65	ReservasController	Concreta	Controlador FXML del módulo de reservas. Gestiona tabla de reservas, formularios y acciones CRUD.
66	HabitacionesController	Concreta	Controlador FXML del módulo de habitaciones. Muestra lista con estado visual y formulario de edición.

67	CientesController	Concreta	Controlador FXML del módulo de clientes. Búsqueda, registro y actualización de clientes con validación en UI.
68	CheckInOutController	Concreta	Controlador FXML del módulo check-in/out. Registra entradas y salidas de huéspedes vinculadas a reservas.
69	FacturacionController	Concreta	Controlador FXML de facturación. Genera facturas, selecciona método de pago y muestra historial.
70	MantenimientoController	Concreta	Controlador FXML de mantenimiento. Registra órdenes, asigna técnicos y actualiza estados.
71	EmpleadosController	Concreta	Controlador FXML del módulo de empleados. Gestiona alta, edición y listado de empleados con sus cargos.
72	ReportesController	Concreta	Controlador FXML de reportes. Filtra por rango de fechas y presenta reportes de ocupación e ingresos.
73	NavigatorUtil	Concreta	Utilitario de navegación entre pantallas FXML. Carga escenas y gestiona el Stage principal centralizado.
74	NotificationUtil	Concreta	Utilitario para mostrar notificaciones visuales en la UI (SUCCESS, ERROR, WARNING, INFO) con JavaFX.
75	MainMenuView	Concreta	Vista de consola del menú principal. Maneja la navegación por teclado en modo texto (fallback sin JavaFX).
76	ReservaView	Concreta	Vista de consola para operaciones de reservas en modo texto.
77	ClienteView	Concreta	Vista de consola para operaciones de clientes en modo texto.
78	HabitacionView	Concreta	Vista de consola para consulta y gestión de habitaciones en modo texto.
79	EmpleadoView	Concreta	Vista de consola para operaciones de empleados en modo texto.
80	FacturaView	Concreta	Vista de consola para consulta y generación de facturas en modo texto.

81	CheckInOutView	Concreta	Vista de consola para registro de check-in y check-out en modo texto.
82	ChatbotView	Concreta	Vista de consola del chatbot. Lee entrada del usuario y muestra respuestas del ChatbotService.
83	AppContext (com)	Concreta	Duplicado en paquete raíz com. Archivo legado del proyecto, reemplazado por com.hotel.AppContext.
84	CheckInOut.fxml	FXML	Vista del módulo de Check-In/Check-Out. Define la interfaz para registrar entradas y salidas de huéspedes.
85	Cientes.fxml	FXML	Vista del módulo de gestión de clientes. Formulario de búsqueda, registro y edición de clientes.
86	Dashboard.fxml	FXML	Vista principal del panel de control. Muestra KPIs, gráficos de ocupación e ingresos del hotel.
87	Empleados.fxml	FXML	Vista del módulo de gestión de empleados. CRUD de empleados con roles y permisos.
88	Facturacion.fxml	FXML	Vista del módulo de facturación. Generación y consulta de facturas con detalles de pago.
89	Habitaciones.fxml	FXML	Vista del módulo de habitaciones. Gestión de estados, tipos y disponibilidad.
90	Login.fxml	FXML	Vista de inicio de sesión. Formulario de autenticación con usuario y contraseña.
91	Mantenimiento.fxml	FXML	Vista del módulo de mantenimiento. Solicitud, seguimiento y cierre de mantenimientos.

92	Reportes.fxml	FXML	Vista del módulo de reportes. Selección de tipo de reporte (ocupación, ingresos diarios, mensuales, anuales).
93	AvatarUtil.java	Concreta	Utilidad UI que genera avatares circulares con iniciales del usuario para mostrar en el Dashboard y menú.
94	BadgeUtil.java	Concreta	Utilidad UI que crea badges/etiquetas de estado con colores (DISPONIBLE=verde, OCUPADA=rojo, etc.).
95	ConfirmDialog.java	Concreta	Componente de diálogo de confirmación reutilizable. Muestra modales de Sí/No antes de acciones destructivas.
96	FacturaTermicaView.java	Concreta	Componente que renderiza la vista previa de factura en formato térmico (tipo ticket de impresora POS).
97	FormField.java	Concreta	Componente reutilizable de campo de formulario con etiqueta, campo de texto y mensaje de validación.
98	ModalUtil.java	Concreta	Utilidad para crear ventanas modales emergentes reutilizables en toda la aplicación JavaFX.
99	NotificationUtil.java	Concreta	Sistema de notificaciones toast. Muestra mensajes de éxito, error y advertencia que se auto-ocultan.
100	ValidacionCampo.java	Concreta	Componente de validación visual en tiempo real para campos de formulario con indicadores de error.
101	main.css	CSS	Hoja de estilos principal de la aplicación. Define la paleta de colores, tipografía y estilos globales de la UI.
102	Auditoria.java	Modelo	Modelo de auditoría del sistema. Registra acciones de usuarios (login, CRUD) con fecha, usuario y detalle.

103	module-info.java	Módulo	Descriptor de módulo Java (JPMS). Declara las dependencias de JavaFX, exports de paquetes y opens para FXML.
104	GeminiApiService.java	Concreta	Servicio de integración con la API de Google Gemini. Envía prompts y procesa respuestas del modelo de IA.
105	AuditoriaService.java	Concreta	Servicio de auditoría. Registra y consulta logs de acciones del sistema delegando al DAO de auditoría.
106	AuditoriaDAOImpl.java	Concreta	Implementación DAO de auditoría. Ejecuta INSERT y SELECT sobre la tabla AUDITORIA en Oracle usando JDBC.

2.2 Definición de Atributos, Métodos y Responsabilidades

Clase: Persona [Abstracta]

Clase:	Persona
Tipo:	Abstracta
Responsabilidad:	Clase base que centraliza atributos de identidad personal comunes a Cliente y Empleado, evitando duplicación de código.

Atributos:

Atributo	Tipo de Dato	Descripción
id	int	Identificador único en base de datos
nombre	String	Primer nombre
apellido	String	Apellido

email	String	Correo electrónico
telefono	String	Teléfono de contacto

Métodos:

Método	Tipo Retorno	Descripción
obtenerNombreCompleto()	String	Retorna nombre y apellido concatenados
getters / setters	varios	Acceso controlado a atributos privados

Clase: Cliente [Concreta]

Clase:	Cliente
Tipo:	Concreta
Responsabilidad:	Extiende Persona para representar al huésped del hotel con información documental y estado VIP.

Atributos:

Atributo	Tipo de Dato	Descripción
documento	String	Número de documento de identidad
nacionalidad	String	País de origen
fechaRegistro	LocalDate	Fecha de registro en el sistema
esVip	boolean	Indica si aplica descuentos VIP
subtipoDocumento	String	COLOMBIANA o EXTRANJERA
ciudadOrigen	String	Ciudad de procedencia del huésped

Métodos:

Método	Tipo Retorno	Descripción
getters / setters	varios	Acceso controlado a todos los atributos

Clase: Empleado [Concreta]

Clase:	Empleado
Tipo:	Concreta
Responsabilidad:	Extiende Persona para representar al personal del hotel con su cargo y fecha de contratación.

Atributos:

Atributo	Tipo de Dato	Descripción
cargo	Cargo	Cargo laboral asignado al empleado
fechaContratacion	LocalDate	Fecha en que fue contratado

Métodos:

Método	Tipo Retorno	Descripción
getters / setters	varios	Acceso controlado a atributos

Clase: Habitacion [Concreta]

Clase:	Habitacion
Tipo:	Concreta
Responsabilidad:	Entidad física del hotel. GRASP Experto: es la única responsable de gestionar su propio estado.

Atributos:

Atributo	Tipo de Dato	Descripción
id	int	Identificador único
numero	String	Número visible (ej: 101)
tipoHabitacion	TipoHabitacion	Tipo al que pertenece
piso	Piso	Piso donde se ubica
estado	EstadoHabitacion	DISPONIBLE, OCUPADA, MANTENIMIENTO, FUERA_DE_SERVICIO
precioBase	double	Precio base en COP
numCamas	int	Número de camas disponibles

Métodos:

Método	Tipo Retorno	Descripción
estaDisponible()	boolean	Verifica si el estado es DISPONIBLE
ocupar()	void	Transiciona a OCUPADA
liberar()	void	Transiciona a DISPONIBLE
enviarAMantenimiento()	void	Transiciona a MANTENIMIENTO
calcularPrecioFinal()	double	Delega a TipoHabitacion el cálculo del precio

Clase: TipoHabitacion [Abstracta]

Clase:	TipoHabitacion
--------	----------------

Tipo:	Abstracta
Responsabilidad:	Define jerarquía de tipos de habitación. Aplica GRASP Polimorfismo y SOLID OCP.

Atributos:

Atributo	Tipo de Dato	Descripción
id	int	ID del tipo
nombre	String	Nombre del tipo
capacidadMaxima	int	Máximo de personas
descripcion	String	Descripción del tipo
amenities	String	Comodidades incluidas

Métodos:

Método	Tipo Retorno	Descripción
calcularPrecio(double)	double	(Abstracto) Aplica multiplicador al precio base
obtenerEtiquetaTipo()	String	(Abstracto) Retorna etiqueta para la UI

Clase: Reserva [Concreta]

Clase:	Reserva
Tipo:	Concreta
Responsabilidad:	Asocia Cliente con Habitacion en rango de fechas. Gestiona su propio ciclo de vida.

Atributos:

Atributo	Tipo de Dato	Descripción
id	int	Identificador único
cliente	Cliente	Huésped que realiza la reserva
habitacion	Habitacion	Habitación reservada
fechaEntrada	LocalDate	Fecha de ingreso planificada
fechaSalida	LocalDate	Fecha de salida planificada
estado	EstadoReserva	PENDIENTE, CONFIRMADA, EN_PROCESO, COMPLETADA, CANCELADA
numPersonas	int	Número de huéspedes
precioTotal	double	Total calculado de la estadía

Métodos:

Método	Tipo Retorno	Descripción
calcularTotalDiasReserva()	long	Días entre entrada y salida
estaActiva()	boolean	Verifica si no está cancelada/completada
confirmar()	void	Transiciona a CONFIRMADA
cancelar()	void	Transiciona a CANCELADA
completar()	void	Transiciona a COMPLETADA
iniciarEstadia()	void	Transiciona a EN_PROCESO

Clase: Factura [Concreta]

Clase:	Factura
--------	---------

Tipo:	Concreta
Responsabilidad:	Documento de cobro generado a partir de reservas completadas. Aplica GRASP Experto calculando su propio total.

Atributos:

Atributo	Tipo de Dato	Descripción
id	int	Identificador único
reserva	Reserva	Reserva asociada
cliente	Cliente	Cliente facturado
fechaEmision	LocalDate	Fecha de emisión
subtotal	double	Precio antes de IVA
tasaIva	double	Tasa de IVA aplicada (0.19)
impuestos	double	Monto de impuestos calculado
total	double	Total con IVA incluido
estadoPago	EstadoPago	PENDIENTE, PAGADA, ANULADA
metodoPago	MetodoPago	EFFECTIVO, TARJETA_CREDITO, TARJETA_DEBITO, TRANSFERENCIA

Métodos:

Método	Tipo Retorno	Descripción
estaPagada()	boolean	Verifica si el estado es PAGADA
getTasaIva()	double	Retorna la tasa IVA almacenada

getters / setters	varios	Acceso a todos los atributos
-------------------	--------	------------------------------

Clase: CheckInOut [Concreta]

Clase:	CheckInOut
Tipo:	Concreta
Responsabilidad:	Registra el ingreso y salida efectiva del huésped con fecha/hora real y empleado responsable.

Atributos:

Atributo	Tipo de Dato	Descripción
id	int	Identificador único
reserva	Reserva	Reserva asociada al registro
empleadoResponsable	Empleado	Empleado que realizó el proceso
fechaHoraCheckin	LocalDateTime	Fecha y hora real de ingreso
fechaHoraCheckout	LocalDateTime	Fecha y hora real de salida (null si aún no salió)
observaciones	String	Notas adicionales del proceso

Métodos:

Método	Tipo Retorno	Descripción
haRealizadoCheckout()	boolean	Verifica si ya se registró la salida
getters / setters	varios	Acceso controlado a atributos

Clase: Mantenimiento [Concreta]

Clase:	Mantenimiento
Tipo:	Concreta
Responsabilidad:	Orden de mantenimiento sobre una habitación con tipo, estado y empleado responsable.

Atributos:

Atributo	Tipo de Dato	Descripción
id	int	Identificador único
habitacion	Habitacion	Habitación en mantenimiento
empleadoResponsable	Empleado	Técnico asignado
fechaSolicitud	LocalDate	Fecha de la solicitud
fechaRealizacion	LocalDate	Fecha de realización (puede ser null)
tipo	TipoMantenimiento	PREVENTIVO, CORRECTIVO, EMERGENCIA
estado	EstadoMantenimiento	SOLICITADO, EN_PROCESO, COMPLETADO, CANCELADO
costo	double	Costo del mantenimiento
descripcionTrabajo	String	Descripción del trabajo realizado

Métodos:

Método	Tipo Retorno	Descripción
estaCompletrado()	boolean	Verifica si el estado es COMPLETADO
getters / setters	varios	Acceso a todos los atributos

Clase: Cargo [Concreta]

Clase:	Cargo
Tipo:	Concreta
Responsabilidad:	Cargo laboral del empleado con nombre, descripción y salario base.

Atributos:

Atributo	Tipo de Dato	Descripción
id	int	Identificador único
nombreCargo	String	Nombre del cargo (ej: Recepcionista)
descripcion	String	Descripción del cargo
salarioBase	double	Salario base en COP

Métodos:

Método	Tipo Retorno	Descripción
getters / setters	varios	Acceso controlado a todos los atributos

Clase: Piso [Concreta]

Clase:	Piso
Tipo:	Concreta
Responsabilidad:	Piso físico del hotel. Referenciado por Habitación para organización física del establecimiento.

Atributos:

Atributo	Tipo de Dato	Descripción
id	int	Identificador único
numeroPiso	int	Número del piso (1, 2, 3...)
descripcion	String	Descripción del piso
cantidadHabitaciones	int	Total de habitaciones en el piso

Métodos:

Método	Tipo Retorno	Descripción
getters / setters	varios	Acceso controlado a todos los atributos

Clase: AuthService [Concreta]

Clase:	AuthService
Tipo:	Concreta
Responsabilidad:	Gestiona autenticación con tokens, roles y expiración de sesiones (8 horas). Aplica SOLID SRP.

Atributos:

Atributo	Tipo de Dato	Descripción
PERMISOS	Map<String,String[]>	Mapa de roles y sus permisos permitidos
sesionesActivas	Map<String,SesionActiva>	Sesiones vigentes indexadas por token
MAX_INTENTOS_FALLIDOS	int	Máximo 3 intentos de login fallidos
MINUTOS_EXPIRACION_SESION	long	480 minutos (8 horas) de vida del token

Métodos:

Método	Tipo Retorno	Descripción
autenticar(email,pass)	String	Valida credenciales y retorna token UUID
cerrarSesion(token)	void	Invalida el token de sesión
verificarPermiso(token,permiso)	boolean	Valida si el token tiene el permiso dado
obtenerEmpleadoDeToken(token)	Optional<Empleado>	Retorna el empleado asociado al token activo
limpiarSesionesExpiradas()	void	Elimina sesiones vencidas del mapa en memoria

Clase: ReservaService [Concreta]

Clase:	ReservaService
Tipo:	Concreta
Responsabilidad:	Controlador GRASP que orquesta el caso de uso de reservas. Aplica SOLID SRP y DIP.

Atributos:

Atributo	Tipo de Dato	Descripción
reservaDAO	IReservaDAO	Acceso CRUD de reservas
reservaBusqueda	IReservaBusqueda	Búsquedas avanzadas de reservas
habitacionDAO	IHabitacionDAO	Acceso a habitaciones
clienteDAO	IClienteDAO	Acceso a clientes
emailService	EmailService	Notificaciones por correo

Métodos:

Método	Tipo Retorno	Descripción
crearReserva(ReservaDTO)	Reserva	Crea reserva en estado PENDIENTE con validaciones de negocio
confirmarReserva(int)	void	Confirma reserva PENDIENTE
cancelarReserva(int)	void	Cancela reserva y libera habitación si estaba OCUPADA
buscarDisponibilidad(DTO)	List<Habitacion>	Busca habitaciones disponibles para fechas dadas

Clase: DatabaseConnection [Concreta]

Clase:	DatabaseConnection
Tipo:	Concreta
Responsabilidad:	Singleton con pool de conexiones JDBC a Oracle. Inicialización LAZY para no bloquear JavaFX.

Atributos:

Atributo	Tipo de Dato	Descripción
instanciaUnica	DatabaseConnection	Referencia única (patrón Singleton, volatile)
pool	BlockingQueue<Connection>	Cola bloqueante con 5 conexiones Oracle
URL_CONEXION	String	jdbc:oracle:thin:@localhost:1521:XE
USUARIO_BD	String	Usuario Oracle (SYSTEM)
TAMANO_POOL	int	Capacidad del pool: 5 conexiones

Métodos:

Método	Tipo Retorno	Descripción
obtenerInstancia()	DatabaseConnection	Retorna la instancia única (double-checked locking)
obtenerConexion()	Connection	Toma una conexión del pool (espera hasta TIMEOUT)
liberarConexion(conn)	void	Devuelve la conexión al pool para reutilización
cerrarPool()	void	Cierra todas las conexiones al apagar la aplicación

Clase: ValidadorEntradas [Concreta]

Clase:	ValidadorEntradas
Tipo:	Concreta
Responsabilidad:	GRASP Fabricación Pura. Centraliza validaciones reutilizables evitando duplicación en los servicios.

Atributos:

Atributo	Tipo de Dato	Descripción
PATRON_EMAIL	Pattern	Regex para validar formato de email
PATRON_TELEFONO	Pattern	Regex para validar formato de teléfono
LARGO_MINIMO_NOMBRE	int	Mínimo 2 caracteres para nombres
LARGO_MAXIMO_NOMBRE	int	Máximo 100 caracteres para nombres

Métodos:

Método	Tipo Retorno	Descripción
validarCampoRequerido(val,campo)	void	Lanza ValidationException si el campo está vacío

validarFormatoEmail(email)	void	Lanza ValidationException si el formato es inválido
validarFormatoTelefono(tel)	void	Valida formato de número telefónico
validarIdPositivo(id,campo)	void	Lanza ValidationException si el ID es <= 0
validarRangoFechas(entrada,salida)	void	Valida que la fecha de salida sea posterior a la de entrada

Clase: NotificationUtil [Concreta]

Clase:	NotificationUtil
Tipo:	Concreta
Responsabilidad:	Utilitario JavaFX para mostrar notificaciones visuales en la UI con tipos SUCCESS, ERROR, WARNING e INFO.

Atributos:

Atributo	Tipo de Dato	Descripción
Tipo	enum	SUCCESS, ERROR, WARNING, INFO

Métodos:

Método	Tipo Retorno	Descripción
mostrar(tipo,mensaje)	void	Muestra notificación visual temporal en la UI
mostrarError(mensaje)	void	Atajo para notificación de tipo ERROR
mostrarExito(mensaje)	void	Atajo para notificación de tipo SUCCESS

Clase: NavigatorUtil [Concreta]

Clase:	NavigatorUtil
Tipo:	Concreta
Responsabilidad:	Centraliza la navegación entre pantallas FXML. Gestiona la carga de escenas en el Stage principal.

Atributos:

Atributo	Tipo de Dato	Descripción
primaryStage	Stage	Referencia al Stage principal de la aplicación

Métodos:

Método	Tipo Retorno	Descripción
navegarA(fxml)	void	Carga el archivo FXML dado y lo muestra en el Stage principal
navegarConControlador(fxml)	Object	Carga FXML y retorna el controlador para configuración adicional

2.3 Relaciones entre Clases

Clase Origen	Clase Destino	Relación	Descripción
Cliente	Persona	Herencia	Cliente hereda atributos de identidad personal de Persona. Aplica SOLID LSP.
Empleado	Persona	Herencia	Empleado hereda de Persona. Sustituible en cualquier contexto que use Persona.

HabitacionSimple	TipoHabitacion	Herencia	Implementa calcularPrecio() con multiplicador 1.0 para habitación individual.
HabitacionDoble	TipoHabitacion	Herencia	Implementa calcularPrecio() para habitación de dos personas.
HabitacionSuite	TipoHabitacion	Herencia	Implementa calcularPrecio() para suite de lujo con amenities premium.
HabitacionMultiple	TipoHabitacion	Herencia	Implementa calcularPrecio() considerando 2 o 3 camas.
HabitacionNoDisponibleException	BusinessException	Herencia	Excepción específica de disponibilidad, extiende la excepción base del negocio.
ValidationException	BusinessException	Herencia	Excepción específica de validación con información del campo inválido.
ReservaNotFoundException	BusinessException	Herencia	Excepción específica cuando no se encuentra una reserva por su ID.
DescuentoVip	Descuento	Implementa	Aplica 15% de descuento para clientes VIP.
DescuentoTemporadaBaja	Descuento	Implementa	Aplica 10% de descuento en temporada baja.
SinDescuento	Descuento	Implementa	Patrón Null Object: descuento 0.0 sin verificaciones null.
ReservaDAOImpl	IReservaDAO / IReservaBusqueda	Implementa	Implementación JDBC concreta del contrato de reservas.
ClienteDAOImpl	IClienteDAO / IClienteBusqueda	Implementa	Implementación JDBC concreta del contrato de clientes.

HabitacionDAOImpl	IHabitacionDAO	Implementa	Implementación JDBC con query de disponibilidad contra reservas activas.
Reserva	Cliente	Composición	Reserva contiene referencia al Cliente que la generó.
Reserva	Habitacion	Composición	Reserva contiene la Habitacion asociada a la estadía.
CheckInOut	Reserva	Composición	CheckInOut referencia la Reserva sobre la que se realiza el proceso.
Factura	Reserva	Composición	Factura referencia la Reserva que origina el cobro.
Habitacion	TipoHabitacion	Agregación	Habitacion usa un TipoHabitacion para delegar el cálculo de precio.
Habitacion	Piso	Agregación	Habitacion pertenece a un Piso del hotel.
Empleado	Cargo	Agregación	Empleado tiene un Cargo asignado que define su rol y salario base.
ReservaService	IReservaDAO	Dependencia	Service depende de la interfaz, no de la implementación JDBC concreta.
BaseDAO	DatabaseConnection	Dependencia	Todos los DAOs acceden al pool de conexiones a través de BaseDAO.

3. Diseño Preliminar de Interfaces Gráficas

El sistema cuenta con 10 pantallas implementadas en JavaFX con archivos FXML:

Pantalla	Descripción y Componentes
----------	---------------------------

Login.fxml	Autenticación con campos usuario/contraseña, validación de credenciales y manejo de intentos fallidos. Diseño sin bordes nativos (StageStyle.UNDECORATED).
Dashboard.fxml	Panel principal con métricas de ocupación, reservas del día y accesos rápidos a todos los módulos del sistema.
Reservas.fxml	Tabla de reservas activas, formulario de nueva reserva, selector de fechas y botones de confirmar/cancelar con validaciones visuales.
Habitaciones.fxml	Lista de habitaciones con estado visual por color, filtros por tipo y piso, y formulario de edición de datos.
Clientes.fxml	Buscador de clientes, formulario de registro con validación de documento, selector de nacionalidad y tipo de documento.
CheckInOut.fxml	Registro de ingreso/salida con fecha/hora real, vinculado a reserva activa del cliente y campo de observaciones.
Facturacion.fxml	Generación de facturas desde reservas completadas, cálculo de IVA, selección de método de pago e historial de facturas.
Mantenimiento.fxml	Registro de órdenes de mantenimiento, asignación de técnico, tipo de trabajo y seguimiento de estado.
Empleados.fxml	Gestión del personal: alta, edición y listado de empleados con cargo asignado y fecha de contratación.
Reportes.fxml	Generación de reportes HTML de ocupación e ingresos filtrados por rango de fechas.

Hotel

NATIVO

Sistema de Gestión Hotelera

Check-in / Check-out

Reservas y Clientes

Facturación y Reportes

HOTEL NATIVO

Ingrese sus credenciales para continuar

CORREO ELECTRÓNICO

CONTRASEÑA

INICIAR SESIÓN

© 2025 Hotel Nativo — Sistema de Gestión

HOTEL NATIVO

Sistema de Gestión

PRINCIPAL

Dashboard

Reservas

Check-in / Check-out

Clientes

RECURSOS

Habitaciones

Maneje de inventario

ADMINISTRACION

Facturación

Reportes

Empleados

Roberto Sanchez

Gerente

Dashboard

Buenas noches, Roberto

Aquí tienes el resumen de hoy en el hotel

28

Total Habitaciones

26

Disponibles

2

Ocupadas

8

Clientes

\$1.61...

Ingresos totales

0

Checkouts hoy

Ocupación de habitaciones

Estado de habitaciones

DISPONIBLE (26)

Ocupada (2)

Reservas mensuales

Reservas por mes

abr

may

Reservas recientes

ID	Cliente	Habitación	Entrada	Total	Estado
----	---------	------------	---------	-------	--------

HOTEL NATIVO

Sistema de Gestión

PRINCIPAL

Dashboard

Reservas

Check-in / Check-out

Clientes

RECURSOS

Habitaciones

Maneje de inventario

ADMINISTRACION

Facturación

Reportes

Empleados

Roberto Sanchez

Gerente

Reservas

Operaciones > Reservas

Buscar reservas, clientes...

último: 09 de mayo 2025

Buscar por cliente, habitación o ID...

Todos

+ Nueva Reserva

✓ Confirmar

✗ Cancelar Reserva

🔄 Refrescar

Total: 6

Pendientes: 3

Confirmadas: 3

En proceso: 2

Completadas: 4

ID	Cliente	Habitación	Entrada	Salida	Noches	Personas	Total	Estado
#41	bukley hgggg	203	2026-05-18	2026-05-20	18	2	\$1,200,000	COMPLETADA
#21	lula martinez	102	2026-04-29	2026-04-30	1	1	\$80,000	EN PROCESO
#23	yoller corzo	104	2026-04-29	2026-05-30	31	1	\$80,000	COMPLETADA
#24	david martinez	107	2026-04-29	2026-04-30	1	1	\$80,000	COMPLETADA
#22	adlan martinez	202	2026-04-29	2026-05-01	2	2	\$240,000	COMPLETADA
#1	jarrison pulgarin	101	2026-04-28	2026-04-30	2	1	\$180,000	EN PROCESO

HOTEL NATIVO

Sistema de Gestión

PRINCIPAL

Dashboard

Reservas

Check-in / Check-out

Clientes

RECURSOS

Habitaciones

Maneje de inventario

ADMINISTRACION

Facturación

Reportes

Empleados

Roberto Sanchez

Gerente

Check-in/out

Operaciones > Check-in

Buscar reservas, clientes...

último: 09 de mayo 2025

Buscar por cliente, habitación o ID...

Check-in

Check-out

Checkouts Auto 11AM

Refrescar

Check-in en proceso: 2

Check-out hoy: 4

ID	Cliente	Habitación	Fecha Entrada	Fecha Salida	Empleado	Hora Checkin	Hora Checkout	Estado
#21	bukley hgggg	203	2026-05-18	2026-05-20	Lucia Diaz	09/05/2026 18...	09/05/2026 18...	COMPLETADA
#5	david martinez	107	2026-04-29	2026-04-30	Jorge Herrera	26/04/2026 20...	26/04/2026 20...	COMPLETADA
#4	yoller corzo	104	2026-04-29	2026-05-30	Jorge Herrera	26/04/2026 18...	26/04/2026 18...	COMPLETADA
#3	jarrison pulgarin	101	2026-04-28	2026-04-30	Jorge Herrera	26/04/2026 17...	—	EN PROCESO
#2	adlan martinez	202	2026-04-29	2026-05-01	Jorge Herrera	26/04/2026 17...	26/04/2026 17...	COMPLETADA
#1	lula martinez	102	2026-04-29	2026-04-30	Jorge Herrera	26/04/2026 13...	—	EN PROCESO

HOTEL NATIVO

Centro de Gestión

PRINCIPAL

Dashboard

OPERACIONES

Reservas

Check-in / Check-out

Cientes

RECURSOS

Habitaciones

Mantenimiento

ADMINISTRACION

Facturación

Reportes

Empleados

Roberto Sanchez

Gestión

Cientes

Operaciones > Cientes

Buscar reserva, cliente...

sábado, 09 de mayo 2026

Buscar por nombre, email o documento

+ Nuevo Cliente

Editar

Eliminar

Marcar VIP

Refrescar

Total: 8

VIP: 1

ID	Nombre	Apellido	Email	Teléfono	Documento	Nacionalidad	VIP	Registro
#02	yolier	corzo	yolier@gmail.com	305335215	454568	colombiana	—	2026-04-28
#2	Ana	GAmez	ana@email.com	3108676543	87654321	Colombiana	—	2026-04-26
#81	bubby	hgghgg	rgnetofgcjdhfjfd.com	766384763	10386452	colobia	—	2026-05-09
#61	adan	martinez	pulgarvadan170@gmail...	3107452743	77199394	colombiana	—	2026-04-28
#63	david	martinez	davidmartinez@gmail.com	3127732743	1066290	colombiana	—	2026-04-28
#41	luis	martinez	luismartinez@gmail.com	3014432743	1067602182	colombiana	—	2026-04-27
#1	Carlos	PÁmez	carlos@email.com	3001234567	12345678	Colombiana	—	2026-04-26
#21	janison	pulgarin	cuentajanison@gmail.com	3127732743	1065280183	colombiana	—	2026-04-27

HOTEL NATIVO

Centro de Gestión

PRINCIPAL

Dashboard

OPERACIONES

Reservas

Check-in / Check-out

Cientes

RECURSOS

Habitaciones

Mantenimiento

ADMINISTRACION

Facturación

Reportes

Empleados

Roberto Sanchez

Gestión

Mantenimiento

Reservas > Mantenimiento

Buscar reserva, cliente...

sábado, 09 de mayo 2026

Buscar por habitación, empleado o tipo

Nuevo

Completar

Cancelar

Refrescar

Pendientes: 0

ID	Habitación	Empleado	Tipo	Estado	Solicitud	Realización	Costo	Descripción
----	------------	----------	------	--------	-----------	-------------	-------	-------------

Sin mantenimientos pendientes

HOTEL NATIVO

Centro de Gestión

PRINCIPAL

Dashboard

OPERACIONES

Reservas

Check-in / Check-out

Cientes

RECURSOS

Habitaciones

Mantenimiento

ADMINISTRACION

Facturación

Reportes

Empleados

Roberto Sanchez

Gestión

Habitaciones

Reservas > Habitaciones

Buscar reserva, cliente...

sábado, 09 de mayo 2026

Todas

Disponibles

Ocupadas

Mantenimiento

Nueva Habitación

Refrescar

Total: 25

Disponibles: 20

Ocupadas: 2

Mantenimiento: 0

Ocupadas: 7%

Hab. 101 SMPL Pos: 1 \$80.000/noche Ocupado	Hab. 102 SMPL Pos: 1 \$80.000/noche Ocupado	Hab. 103 SMPL Pos: 1 \$80.000/noche Disponible	Hab. 104 SMPL Pos: 1 \$80.000/noche Disponible	Hab. 105 SMPL Pos: 1 \$80.000/noche Disponible
Hab. 106 SMPL Pos: 1 \$80.000/noche Disponible	Hab. 107 SMPL Pos: 1 \$80.000/noche Disponible	Hab. 201 MULTIPLE Pos: 2 \$120.000/noche Disponible	Hab. 202 MULTIPLE Pos: 2 \$120.000/noche Disponible	Hab. 203 MULTIPLE Pos: 2 \$120.000/noche Disponible
Hab. 204 MULTIPLE Pos: 2 \$120.000/noche Disponible	Hab. 205 MULTIPLE Pos: 2 \$180.000/noche Disponible	Hab. 206 MULTIPLE Pos: 2 \$180.000/noche Disponible	Hab. 207 MULTIPLE Pos: 2 \$180.000/noche Disponible	Hab. 301 DOB Pos: 3 \$180.000/noche Disponible

HOTEL NATIVO

Centro de Gestión

PRINCIPAL

Dashboard

OPERACIONES

Reservas

Check-in / Check-out

Cientes

RECURSOS

Habitaciones

Mantenimiento

ADMINISTRACION

Facturación

Reportes

Empleados

Roberto Sanchez

Gestión

Facturación

Admin > Facturación

Buscar reserva, cliente...

sábado, 09 de mayo 2026

Generar Factura

Ver HTML

Refrescar

Total: 3

Pagados: 3

ID	Cliente	Reserva	Fecha Emisión	Subtotal	Impuestos	Total	Estado Pago	Método Pago
#21	bubby hgghgg	#41	2026-05-09	\$1,200,000	\$28,000	\$1,428,000	PAGADA	TARJETA DEBITO
#1	yolier corzo	#23	2026-04-28	\$80,000	\$15,200	\$95,200	PAGADA	EFFECTIVO
#2	david martinez	#24	2026-04-28	\$80,000	\$15,200	\$95,200	PAGADA	EFFECTIVO

HOTEL NATIVO

Centro de Gestión

PRINCIPAL

Dashboard

OPERACIONES

Reservas

Check-in / Check-out

Cientes

RECURSOS

Habitaciones

Mantenimiento

ADMINISTRACION

Facturación

Reportes

Empleados

Roberto Sanchez

Gestión

Reportes

Admin > Reportes

Buscar reserva, cliente...

sábado, 09 de mayo 2026

Reportes

Genera y exporta reportes del sistema en formato HTML

Factura HTML

Genera el archivo HTML de una factura existente por su ID

ID Factura: 1

Generar Factura HTML

Reporte de Ocupación Mensual

Genera el reporte de ocupación de habitaciones para un mes específico

Año: 2025

Mes: 1-12

Generar Reporte Ocupación

HOTEL NATIVO

Centro de Gestión

PRINCIPAL

Dashboard

OPERACIONES

Reservas

Check-in / Check-out

Cientes

RECURSOS

Habitaciones

Mantenimiento

ADMINISTRACION

Facturación

Reportes

Empleados

Roberto Sanchez

Gestión

Empleados

Admin > Empleados

Buscar reserva, cliente...

sábado, 09 de mayo 2026

Nuevo Empleado

Editar

Eliminar

Refrescar

Total: 14

ID	Nombre	Apellido	Email	Teléfono	Cargo	Contratación
#24	Laura	Castro	laura@hotelnativo.com	308444444	Recepcionista	2026-04-27
#51	Fernando	Cruz	fernando@hotelnativo.com	906111111	Servicio Alir Accede...	2026-04-27
#80	Luis	Uster	luis@hotelnativo.com	902555555	Auxiliador	2026-04-27
#21	Jorge	Hernan	jorge@hotelnativo.com	302111111	Vigilante	2026-04-27
#41	Andrea	Logan	andrea@hotelnativo.com	302111111	Recepcionista	2026-04-27
#26	Andres	Medina	andres@hotelnativo.com	306222222	Decorador	2026-04-27
#36	Cecilia	Ortega	cecilia@hotelnativo.com	308444444	Tecnico Sistemas	2026-04-27
#28	Rosa	Pedraza	rosa@hotelnativo.com	302111111	Asesora	2026-04-27
#37	Oscar	Rivera	oscar@hotelnativo.com	906555555	Pintor	2026-04-27
#22	Carlos	Ramirez	carlos@hotelnativo.com	306222222	Administrador	2026-04-27
#34	Alejandra	Rios	alejandra@hotelnativo.com	302111111	Contador	2026-04-27

4. Aplicación de Principios de Diseño Orientado a Objetos

4.1 Patrones GRASP

Patrón	Clase donde se aplica	Razón de diseño
Experto en Información	Habitacion	Gestiona su propio estado (ocupar, liberar, enviarAMantenimiento). Nadie externo modifica el estado directamente.
Experto en Información	Reserva	Conoce sus fechas y calcula calcularTotalDiasReserva() sin depender de otra clase.
Experto en Información	Factura	Calcula su propio total (subtotal + IVA) en el constructor. No necesita un servicio externo para eso.
Controlador	ReservaService	Orquesta el caso de uso 'Realizar Reserva' coordinando DAOs sin mezclar lógica de presentación.
Controlador	CheckInOutService	Gestiona el flujo completo de check-in y checkout automático.
Controlador	AuthService	Controla el ciclo de vida de sesiones: autenticación, permisos y expiración.
Creador	ReservaService	Tiene los datos necesarios (cliente, habitación, fechas) para crear instancias de Reserva.
Creador	AppContext	Instancia y conecta todos los servicios y DAOs al arrancar la aplicación.
Polimorfismo	TipoHabitacion (subclases)	calcularPrecio() se implementa diferente en Simple, Doble, Suite y Multiple sin condicionales if/switch.
Polimorfismo	Descuento (implementaciones)	calcularDescuento() varía en VIP, TemporadaBaja y SinDescuento sin código condicional en ReservaService.

Bajo Acoplamiento	Services → Interfaces DAO	Todos los servicios dependen de interfaces, no de implementaciones JDBC. Cambiar la BD no afecta la lógica.
Alta Cohesión	Todos los Services	Cada service solo se ocupa de su entidad (ReservaService = reservas, ClienteService = clientes, etc.).
Indirección	Capa DAO	Las interfaces DAO desacoplan servicios de la implementación de base de datos.
Fabricación Pura	ValidadorEntradas	No existe en el dominio, pero centraliza validaciones reutilizables para evitar duplicación.
Fabricación Pura	DatabaseConnection	No es del dominio hotelero, pero centraliza la gestión del pool de conexiones.

4.2 Principios SOLID

Principio	Clase / Relación	Justificación
SRP	ReservaService	Solo gestiona lógica de reservas. No mezcla acceso a datos ni presentación.
SRP	EmailService	Responsabilidad única: envío de notificaciones. MantenimientoService lo delega en lugar de hacer println.
SRP	ValidadorEntradas	Solo valida datos de entrada. No tiene otra responsabilidad.
OCP	TipoHabitacion y subclases	Para agregar un tipo nuevo se crea una subclase sin tocar el código existente.
OCP	Descuento y sus implementaciones	Nuevas estrategias de descuento se agregan implementando la interfaz, sin modificar ReservaService.
LSP	Cliente y Empleado extienden Persona	Cualquier código que use Persona puede recibir Cliente o Empleado sin alterarse.

ISP	IReservaDAO / IReservaBusqueda	CRUD y búsquedas avanzadas están en interfaces separadas. Nadie implementa métodos que no necesita.
ISP	IClienteDAO / IClienteBusqueda	Segregación idéntica: CRUD separado de búsquedas por nombre, documento y email.
DIP	ReservaService → IReservaDAO	Service depende de la abstracción. Si se cambia Oracle por otra BD, solo cambia la implementación concreta.
DIP	ApplicationContext	Instancia implementaciones concretas e inyecta interfaces a los servicios. Centraliza la composición.

5. Gestión del Proyecto con Git

5.1 Repositorio en GitHub

Campo	Detalle
Nombre	hotel-system-nativo
URL	https://github.com/[usuario]/hotel-system-nativo
Descripción	Sistema de gestión hotelera en Java con JavaFX y Oracle. Proyecto de aula — Programación III, Universidad Popular del Cesar.
Rama principal	main
Visibilidad	Privado (acceso compartido con el docente)

5.2 Roles del Equipo

Integrante	Rol	Responsabilidades
Jarrison Pulgarín	Líder del Proyecto	Coordinación general, revisión de integraciones, gestión de la rama main, merge de ramas y resolución de conflictos. Capa de servicios.

Roberto Buelvas	Desarrollador Backend	Capa de acceso a datos (DAOs), conexión Oracle, CRUD y manejo de excepciones de BD.
Juan Buelvas	Desarrollador Frontend	Interfaces gráficas FXML, controladores JavaFX, estilos CSS y vinculación UI con servicios.

5.3 Estrategia de Ramas

Rama	Responsable	Descripción
main	Jarrison Pulgarín (Líder)	Rama estable. Solo recibe merges revisados y aprobados por el líder.
feature/jarrison-services	Jarrison Pulgarín	Capa de servicios: ReservaService, ClienteService, HabitacionService, AuthService, ApplicationContext.
feature/roberto-dao	Roberto Buelvas	DAOs con JDBC: ClienteDAOImpl, ReservaDAOImpl, HabitacionDAOImpl, conexión Oracle y esquema SQL.
feature/juan-ui	Juan Buelvas	Pantallas FXML, controladores JavaFX, estilos CSS y utilitarios de navegación.
hotfix/[descripcion]	Cualquier integrante	Correcciones urgentes de errores críticos, creadas desde main y fusionadas directamente.

5.4 Tabla de Tareas por Integrante

Tarea	Integrante	Rama	Estado
Implementar ReservaService (completo)	Jarrison Pulgarín	feature/jarrison-services	En progreso
Implementar ClienteService	Jarrison Pulgarín	feature/jarrison-services	En progreso
Implementar HabitacionService	Jarrison Pulgarín	feature/jarrison-services	En progreso

Implementar AuthService (login y sesiones)	Jarrison Pulgarín	feature/jarrison-services	Completado
Implementar ApplicationContext (composición)	Jarrison Pulgarín	feature/jarrison-services	Completado
Implementar ChatbotService (NLP básico)	Jarrison Pulgarín	feature/jarrison-services	Completado
Implementar ReservaDAOImpl	Roberto Buelvas	feature/roberto-dao	En progreso
Implementar ClienteDAOImpl	Roberto Buelvas	feature/roberto-dao	En progreso
Implementar HabitacionDAOImpl	Roberto Buelvas	feature/roberto-dao	En progreso
Implementar CheckInOutDAOImpl	Roberto Buelvas	feature/roberto-dao	En progreso
Implementar FacturaDAOImpl	Roberto Buelvas	feature/roberto-dao	Pendiente
Implementar MantenimientoDAOImpl	Roberto Buelvas	feature/roberto-dao	Pendiente
Configurar DatabaseConnection (pool)	Roberto Buelvas	feature/roberto-dao	Completado
Crear esquema SQL Oracle	Roberto Buelvas	feature/roberto-dao	Completado
Diseñar Login.fxml y LoginController	Juan Buelvas	feature/juan-ui	Completado
Diseñar Dashboard.fxml y DashboardController	Juan Buelvas	feature/juan-ui	Completado
Diseñar Reservas.fxml y ReservasController	Juan Buelvas	feature/juan-ui	En progreso
Diseñar Habitaciones.fxml y HabitacionesController	Juan Buelvas	feature/juan-ui	En progreso
Diseñar Clientes.fxml y ClientesController	Juan Buelvas	feature/juan-ui	En progreso
Diseñar CheckInOut.fxml y CheckInOutController	Juan Buelvas	feature/juan-ui	Pendiente

Diseñar Facturacion.fxml y FacturacionController	Juan Buelvas	feature/juan-ui	Pendiente
Diseñar Mantenimiento.fxml y MantenimientoController	Juan Buelvas	feature/juan-ui	Pendiente
Diseñar Empleados.fxml y EmpleadosController	Juan Buelvas	feature/juan-ui	Pendiente
Implementar NavigatorUtil y NotificationUtil	Juan Buelvas	feature/juan-ui	Completado
Documentación Fase 2	Jarrison Pulgarín	main	Completado

FASE 3

1. CLASES BÁSICAS

1.1 Descripción General del Modelo de Dominio

El sistema hotelero implementa un conjunto de clases del dominio que representan las entidades del negocio real: huéspedes, habitaciones, reservas, facturas, empleados y mantenimiento. Todas las clases siguen las convenciones de nomenclatura Java (PascalCase para clases, camelCase para atributos y métodos) y aplican el principio de encapsulamiento mediante modificadores de acceso private con exposición a través de getters y setters.

1.2 Clases del Modelo Implementadas

Clase	Paquete	Tipo	Responsabilidad Principal
Persona	com.hotel.model	Clase abstracta	Base de Cliente y Empleado; agrupa atributos comunes de toda persona
Cliente	com.hotel.model	Clase concreta	Representa al huésped con documento, nacionalidad, ciudad de origen y estado VIP
Empleado	com.hotel.model	Clase concreta	Modela al trabajador con cargo, tipo de contrato, salario y credenciales de acceso
Habitacion	com.hotel.model	Clase concreta	Encapsula número, piso, tipo, precio base y estado de la habitación
Reserva	com.hotel.model	Clase concreta	Gestiona el ciclo de vida de una reserva (fechas, estado, precio total, canal)
Factura	com.hotel.model	Clase concreta	Registra el comprobante de pago con subtotal, IVA, método de pago y estado
CheckInOut	com.hotel.model	Clase concreta	Almacena las marcas de tiempo de entrada y salida del huésped
Mantenimiento	com.hotel.model	Clase concreta	Gestiona órdenes de trabajo de mantenimiento con tipo, costo y estado

Clase	Paquete	Tipo	Responsabilidad Principal
Cargo	com.hotel.model	Clase concreta	Catálogo de cargos laborales con salario base
Piso	com.hotel.model	Clase concreta	Representa los pisos del hotel con cantidad de habitaciones
Auditoria	com.hotel.model	Clase concreta	Registro inmutable de acciones del sistema (quién, qué, cuándo, sobre qué entidad)

1.3 Detalle de Clases Clave

1.3.1 Clase Persona (abstracta)

Clase base abstracta que centraliza los atributos comunes a toda persona dentro del sistema. Declara obtenerNombreCompleto() como método concreto, reutilizable por las subclases sin necesidad de duplicación.

Atributo	Tipo	Visibilidad	Descripción
id	int	private	Identificador único en la base de datos
nombre	String	private	Primer nombre de la persona
apellido	String	private	Primer apellido
email	String	private	Correo electrónico (único)
telefono	String	private	Número de teléfono de contacto

Métodos principales:

- obtenerNombreCompleto() — Concatena nombre y apellido para presentación.
- Getters y setters completos para cada atributo (encapsulamiento).

1.3.2 Clase Cliente

Extiende Persona añadiendo los datos específicos del huésped: documento de identidad, nacionalidad, ciudad de origen y el indicador VIP que activa descuentos automáticos.

Atributo	Tipo	Descripción
segundoNombre	String	Segundo nombre (opcional)
apellido2	String	Segundo apellido
documento	String	Número de documento de identidad

Atributo	Tipo	Descripción
subtipoDocumento	String	Tipo: CC, CE, Pasaporte, etc.
nacionalidad	String	País de nacionalidad
idPais	int	FK a tabla PAIS
idCiudad	int	FK a tabla CIUDAD
ciudadOrigen	String	Ciudad de procedencia
fechaRegistro	LocalDate	Fecha de primera reserva o registro
esVip	boolean	Indicador de cliente VIP (descuento del 15%)

1.3.3 Clase Empleado

Extiende Persona con los atributos laborales y de autenticación. El atributo hashContrasena jamás se almacena en texto plano; siempre se guarda el hash BCrypt con costo 12.

Atributo	Tipo	Descripción
cargo	Cargo	Objeto con el cargo asignado (FK CARGO)
salario	double	Salario mensual del empleado
tipoContrato	String	INDEFINIDO, FIJO, PRESTACION
tipoPago	String	QUINCENAL, MENSUAL
fechaContratacion	LocalDate	Fecha de inicio del contrato
fechaFinContrato	LocalDate	Fecha de terminación (nullable)
hashContrasena	String	Hash BCrypt de la contraseña
debeCambiarContrasena	boolean	Flag para forzar cambio en próximo login

1.3.4 Clase Habitación

Entidad central del sistema. Mantiene su estado mediante un enum propio y expone métodos de transición de estado que encapsulan las reglas del negocio.

Atributo / Método	Tipo	Descripción
numero	String	Número de habitación (ej. "101")
tipoHabitacion	TipoHabitacion	Tipo polimórfico (Simple, Doble, Multiple, Suite)
piso	int	Número de piso
estado	EstadoHabitacion	DISPONIBLE, RESERVADA, OCUPADA, LIMPIEZA, MANTENIMIENTO, FUERA DE SERVICIO
precioBase	double	Precio base por noche (sin descuentos)
numCamas	int	Número de camas disponibles

Atributo / Método	Tipo	Descripción
estaDisponible()	boolean	Verifica si el estado es DISPONIBLE
reservar()	void	Transicion → RESERVADA
ocupar()	void	Transicion → OCUPADA
liberar()	void	Transicion → DISPONIBLE
enviarAMantenimiento()	void	Transicion → MANTENIMIENTO
calcularPrecioFinal()	double	Delega al TipoHabitacion el calculo con multiplicador

1.3.5 Clase Reserva

Gestiona el ciclo completo de una reserva: desde su creación en estado PENDIENTE hasta la finalización en COMPLETADA. Los métodos de transición garantizan que el flujo siga el orden correcto de estados.

Estado	Descripción
PENDIENTE	Recién creada, a la espera de confirmación
CONFIRMADA	Confirmada por el cliente, habitación bloqueada
EN_PROCESO	Cliente realizó check-in
COMPLETADA	Cliente realizó check-out, factura generada
CANCELADA	Reserva anulada por cliente o sistema

1.3.6 Clase Factura

Registra el comprobante de cobro. Distingue el método de pago mediante el enum MetodoPago y almacena los campos específicos de cada método (efectivo: montoRecibido, cambio; tarjeta: franquiciaTarjeta, numCuotas; transferencia: referenciaTransferencia).

Enum MetodoPago	Enum EstadoPago
EFFECTIVO	PENDIENTE
TARJETA_CREDITO	PAGADA
TARJETA_DEBITO	ANULADA
TRANSFERENCIA	—

1.4 Enumeraciones del Sistema

El sistema utiliza enumeraciones (enum) para representar estados finitos, evitando el uso de literales de texto (magic strings) y garantizando type-safety en tiempo de compilación.

Enum	Clase contenedora	Valores
EstadoHabitacion	Habitacion	DISPONIBLE, RESERVADA, OCUPADA, LIMPIEZA, MANTENIMIENTO, FUERA_DE_SERVICIO
EstadoReserva	Reserva	PENDIENTE, CONFIRMADA, EN_PROCESO,

Enum	Clase contenedora	Valores
		COMPLETADA, CANCELADA
EstadoPago	Factura	PENDIENTE, PAGADA, ANULADA
MetodoPago	Factura	EFFECTIVO, TARJETA_CREDITO, TARJETA_DEBITO, TRANSFERENCIA
TipoMantenimiento	Mantenimiento	PREVENTIVO, CORRECTIVO, EMERGENCIA
EstadoMantenimiento	Mantenimiento	SOLICITADO, EN_PROCESO, COMPLETADO, CANCELADO

2.1 Jerarquía de Clases

El sistema implementa herencia en dos jerarquías principales del dominio y una jerarquía técnica en la capa DAO.

2.1.1 Jerarquía de Persona

Persona (abstract)

- └─ Cliente — getters de documento, idPais, esVip, etc.
- └─ Empleado — getters de cargo, hashContraseña, salario, etc.

Persona declara los atributos comunes (id, nombre, apellido, email, telefono) con encapsulamiento private. Las subclases acceden a ellos mediante los getters/setters heredados, respetando el principio de Liskov: cualquier lugar del código que espera una Persona puede recibir un Cliente o un Empleado sin que el comportamiento se rompa.

2.1.2 Jerarquía de TipoHabitacion (Polimorfismo)

TipoHabitacion (abstract)

- └─ HabitacionSimple — MULTIPLICADOR_PRECIO = 1.0
- └─ HabitacionDoble — MULTIPLICADOR_PRECIO = 1.0
- └─ HabitacionMultiple — capacidad variable (2-3 camas)
- └─ HabitacionSuite — MULTIPLICADOR_PRECIO premium (mayor precio)

La clase abstracta TipoHabitacion declara el método abstracto calcularPrecio(double precioBase) y el método concreto obtenerEtiquetaTipo(). Cada subclase sobrescribe calcularPrecio() aplicando su propio multiplicador, lo que permite que la clase Habitacion invoque polimórficamente tipoHabitacion.calcularPrecio(precioBase) sin conocer el tipo concreto en tiempo de compilación.

Subclase	Método sobrescrito	Comportamiento
HabitacionSimple	calcularPrecio(double)	Retorna precioBase x 1.0 (tarifa estándar)
HabitacionDoble	calcularPrecio(double)	Retorna precioBase x 1.0 con configuración para dos personas

Subclase	Método sobrescrito	Comportamiento
HabitacionMultiple	calcularPrecio(double)	Calcula según número de camas configuradas (2 o 3)
HabitacionSuite	calcularPrecio(double)	Aplica multiplicador premium (mayor precio por noche)

2.1.3 Jerarquía de BaseDAO (Herencia Técnica)

BaseDAO (abstract)

- ClienteDAOImpl
- ReservaDAOImpl
- HabitacionDAOImpl
- EmpleadoDAOImpl
- FacturaDAOImpl
- CheckInOutDAOImpl
- MantenimientoDAOImpl
- AuditoriaDAOImpl
- UbicacionDAO

BaseDAO gestiona el pool de conexiones (BlockingQueue<Connection>) y el patrón de transacción mediante el método plantilla enTransaccion(TransaccionCallback). Todas las implementaciones DAO heredan este comportamiento sin repetirlo (reutilización de código, principio DRY).

2.1.4 Jerarquía de Excepciones Personalizadas

RuntimeException

- ExcepcionBaseDatos — errores de la capa de persistencia
- ExcepcionNegocio — violaciones de reglas de negocio
- ExcepcionValidacion — fallos en validacion de entrada
- ExcepcionReservaNoEncontrada
- HabitacionNoDisponibleException

2.2 Interfaces Implementadas

Se utilizan interfaces DAO para definir contratos de acceso a datos, desacoplando la lógica de negocio de la tecnología de persistencia.

Interfaz	Implementada por	Métodos principales
IClienteDAO	ClienteDAOImpl	insertar(), actualizar(), eliminar(), buscarPorId(), listarTodos()
IClienteBusqueda	ClienteDAOImpl	buscarPorEmail(), buscarPorNombre(), buscarVips()
IReservaDAO	ReservaDAOImpl	insertar(), cancelar(), verificarDisponibilidad(), calcularPrecio()
IReservaBusqueda	ReservaDAOImpl	buscarPorCliente(), buscarPorFechas(), buscarActivas()

Interfaz	Implementada por	Métodos principales
IHabitacionDAO	HabitacionDAOImpl	insertar(), listarDisponibles(), buscarPorId()
IEmpleadoDAO	EmpleadoDAOImpl	insertar(), actualizar(), buscarPorCargo()
IFacturaDAO	FacturaDAOImpl	insertar(), buscarPorReserva(), listarTodas()
IMantenimientoDAO	MantenimientoDAOImpl	insertar(), actualizar(), listarTodas()
IAuditoriaDAO	AuditoriaDAOImpl	insertar(), buscarPorRango()
Descuento	DescuentoVip, DescuentoTemporadaBaja, SinDescuento	calcularDescuento(double), obtenerDescripcion()

2.3 Colecciones Utilizadas

Colección	Clase / Paquete	Uso
ArrayList<T>	Todas las impl. DAO	Almacena los resultados de consultas SQL (listarTodos, buscar*). Se elige ArrayList por acceso indexado O(1) y uso secuencial de los resultados.
BlockingQueue<Connection>	BaseDAO, ConexionBaseDatos	Pool de conexiones (5 conexiones). Bloqueo automático cuando el pool está vacío, garantizando thread-safety sin sincronización explícita.
ConcurrentHashMap<String,?>	AuthService	Sesiones activas, credenciales por email e intentos fallidos. Operaciones atómicas sin locks explícitos.
HashMap<String, String[]>	ChatbotService	Mapeo de sinónimos para reconocimiento de intención del chatbot.
Map<String, String[]>	AuthService	Permisos por rol: ADMIN → [todas las acciones], RECEPCIONISTA → [reservas, clientes, checkin], etc.
Optional<T>	Interfaces DAO	Tipo de retorno para búsquedas por ID. Elimina NullPointerException y obliga al llamador a manejar el caso de ausencia.

2.4 Patrón Strategy con Polimorfismo

El sistema aplica el patrón de diseño Strategy para el cálculo de descuentos en reservas. La interfaz Descuento define el contrato, y en tiempo de ejecución ReservaService selecciona la estrategia adecuada según las características del cliente y la época del año.

Estrategia	Descuento aplicado	Condición de activación
DescuentoVip	15% sobre el total	Cliente con esVip = true
DescuentoTemporadaBaja	10% sobre el total	Reserva en período de temporada baja
SinDescuento	0%	Caso por defecto (Null Object)

3.1 Conexión al Motor de Base de Datos

El sistema utiliza Oracle Database XE 21c como motor relacional y se conecta mediante el driver JDBC delgado (thin driver). La gestión de la conexión está centralizada en la clase ConexionBaseDatos, que implementa el patrón Singleton con inicialización lazy y double-checked locking para garantizar una única instancia en entornos multihilo.

Las credenciales de conexión (URL, usuario, contraseña) se cargan desde el archivo externo config/db.properties, excluido del repositorio mediante .gitignore para proteger información sensible.

config/db.properties (no versionado)

db.url=jdbc:oracle:thin:@localhost:1521:xe

db.user=hotel_user

db.password=*****

El pool de conexiones mantiene 5 conexiones activas almacenadas en un BlockingQueue<Connection>. Cuando un hilo requiere una conexión, la toma del pool (bloqueo si está vacío) y la devuelve al finalizar, evitando el costo de abrir una conexión por cada operación.

3.2 Mapeo de Objetos a Tablas (ORM Manual)

Dado que no se utiliza un framework ORM como Hibernate, el mapeo entre objetos Java y tablas Oracle se realiza manualmente en cada implementación DAO mediante PreparedStatement y ResultSet.

Clase del Dominio	Tabla Oracle	Columnas principales mapeadas
Cliente	CLIENTE	id_cliente, primer_nombre, apellido_1, email, telefono, id_pais, id_ciudad, es_vip, tipo_documento, numero_documento
Empleado	EMPLEADO	id_empleado, primer_nombre, apellido_1, email, id_cargo, salario, tipo_contrato, password_hash, debe_cambiar_password
Habitacion	HABITACION	id_habitacion, numero_habitacion,

Clase del Dominio	Tabla Oracle	Columnas principales mapeadas
		tipo_habitacion, id_piso, estado, precio_base, num_camras
Reserva	RESERVA	id_reserva, id_cliente, id_habitacion, fecha_entrada, fecha_salida, estado, num_personas, precio_total, id_canal
Factura	FACTURA	id_factura, id_reserva, subtotal, impuestos, total, estado_pago, metodo_pago, fecha_emision
CheckInOut	CHECKIN_OUT	id_checkin_out, id_reserva, id_empleado, fecha_hora_checkin, fecha_hora_checkout, observaciones
Mantenimiento	MANTENIMIENTO	id_mantenimiento, id_habitacion, tipo, estado, costo, descripcion_trabajo, id_empleado_responsable
Auditoria	AUDITORIA	id_auditoria, fecha_hora, id_empleado, accion, entidad, id_entidad, detalles
Cargo	CARGO	id_cargo, nombre_cargo, descripcion, salario_base
Piso	PISO	id_piso, numero_piso, descripcion, cantidad_habitaciones

Ejemplo de mapeo en ClienteDAOImpl

El método buscarPorId(int id) ejecuta una consulta con PreparedStatement y construye el objeto Cliente a partir del ResultSet:

```
String sql = "SELECT c.*, p.nombre AS pais, ci.nombre AS ciudad " +
    "FROM CLIENTE c " +
    "LEFT JOIN PAIS p ON c.id_pais = p.id_pais " +
    "LEFT JOIN CIUDAD ci ON c.id_ciudad = ci.id_ciudad " +
    "WHERE c.id_cliente = ?";
```

```
PreparedStatement stmt = conn.prepareStatement(sql);
```

```
stmt.setInt(1, id);
```

```
ResultSet rs = stmt.executeQuery();
```

```
if (rs.next()) {
```

```
    Cliente c = new Cliente();
```

```
    c.setId(rs.getInt("id_cliente"));
```

```
    c.setNombre(rs.getString("primer_nombre"));
```

```
    c.setEmail(rs.getString("email"));
```

```

        c.setEsVip(rs.getString("es_vip").equals("S"));
        // ... mas campos
        return Optional.of(c);
    }
    return Optional.empty();

```

3.3 Operaciones CRUD Implementadas

Operación	Método DAO	Mecanismo SQL
Create	insertar(entidad)	Llamada a procedimiento almacenado Oracle (PKG_ADMIN.registrar_cliente, PKG_HOTEL.crear_reserva) o INSERT INTO directo con PreparedStatement
Read	buscarPorId(id), listarTodos()	SELECT con PreparedStatement, paginacion con OFFSET/FETCH en Oracle
Update	actualizar(entidad)	Llamada a procedimiento PKG_ADMIN.actualizar_cliente o UPDATE ... SET directo
Delete	eliminar(id)	DELETE FROM o marcado logico (soft delete) segun la entidad

Adicionalmente, las interfaces de búsqueda (IClienteBusqueda, IReservaBusqueda) exponen operaciones especializadas: búsqueda por email, por rango de fechas, por estado, por cliente VIP, etc.

3.4 Procedimientos Almacenados Oracle

Las operaciones de negocio complejas se delegan a paquetes PL/SQL para aprovechar la lógica del servidor y garantizar la integridad transaccional.

Procedimiento / Función	Paquete Oracle	Invocado desde	Propósito
registrar_cliente()	PKG_ADMIN	ClienteDAOImpl	Inserta cliente validando unicidad de documento y email
actualizar_cliente()	PKG_ADMIN	ClienteDAOImpl	Actualiza datos del cliente con validaciones
crear_reserva()	PKG_HOTEL	ReservaDAOImpl	Crea reserva verificando disponibilidad y solapamiento
cancelar()	PKG_HOTEL	ReservaDAOImpl	Cancela reserva y libera la habitacion

Procedimiento / Función	Paquete Oracle	Invocado desde	Propósito
disponible()	PKG_HOTEL	ReservaDAOImpl	Verifica si una habitacion está libre en un rango de fechas
precio_reserva()	PKG_HOTEL	ReservaDAOImpl	Calcula el precio total con descuento VIP si aplica

3.5 Patrón DAO Aplicado

El patrón Data Access Object (DAO) desacopla la capa de negocio de la capa de acceso a datos. Cada entidad del dominio tiene:

1. Una interfaz que define el contrato (qué se puede hacer).
2. Una implementación que contiene el SQL y la lógica JDBC (cómo se hace).

Esta separación permite que los servicios (ReservaService, ClienteService) dependan de la interfaz y no de la implementación concreta, facilitando el cambio de tecnología de persistencia sin modificar la lógica de negocio (principio de inversión de dependencias).

3.6 Manejo de Excepciones en la Capa de Datos

Toda excepción SQLException lanzada por JDBC se captura en la implementación DAO y se relanza como ExcepcionBaseDatos (no verificada — unchecked), evitando que la firma de los métodos de interfaz incluya "throws SQLException" y simplificando el código en capas superiores.

Excepción personalizada	Cuándo se lanza
ExcepcionBaseDatos	Error de conexion, timeout, violacion de constraint en BD
ExcepcionNegocio	Habitacion ya reservada, fechas invalidas, cliente con deuda
ExcepcionValidacion	Email malformado, documento vacio, fechas en el pasado
ExcepcionReservaNoEncontrada	buscarPorId() retorna Optional.empty() y el llamador lo exige
HabitacionNoDisponibleException	Intento de reservar habitacion en estado OCUPADA o MANTENIMIENTO

3.7 Triggers de Base de Datos

Se implementaron triggers Oracle para garantizar reglas de integridad que no deben depender del cliente Java:

- `trg_no_solapar`: Impide reservas que se superpongan en fechas para la misma habitación.
- `corregir_trigger_vip`: Actualiza automáticamente el indicador `es_vip` cuando un cliente acumula el número de estadías requerido.
- `trg_liberar_mant`: Cambia el estado de la habitación a DISPONIBLE cuando se completa un mantenimiento.

4. INTERFAZ GRÁFICA Y ARQUITECTURA EN TRES CAPAS

4.1 Tecnología Utilizada

La interfaz gráfica está construida con JavaFX 11+ usando el patrón MVC (Model-View-Controller). Las vistas se definen en archivos FXML (XML declarativo) procesados por FXXMLLoader, y cada vista tiene un controlador Java asociado que gestiona los eventos de usuario e interactúa con la capa de servicio.

4.2 Ventanas Implementadas

Pantalla	Archivo FXML	Controlador	Función
Login	Login.fxml	LoginController	Autenticación con email/contraseña, cambio de contraseña obligatorio en primer acceso
Dashboard	Dashboard.fxml	DashboardController	Panel principal con estadísticas de ocupación, ingresos y alertas
Clientes	Clientes.fxml	ClientesController	CRUD de clientes, búsqueda por nombre/documento, indicador VIP
Reservas	Reservas.fxml	ReservasController	Gestión de reservas con calendario, filtros de estado y listado
Habitaciones	Habitaciones.fxml	HabitacionesController	Inventario de habitaciones con indicadores de estado en color
Check-In/Out	CheckInOut.fxml	CheckInOutController	Registro de llegada y salida de huéspedes con marca de tiempo
Facturación	Facturacion.fxml	FacturacionController	Generación de facturas, selección de método de pago, cálculo de cambio
Empleados	Empleados.fxml	EmpleadosController	Directorio de empleados, asignación de cargos y contratos
Mantenimiento	Mantenimiento.fxml	MantenimientoController	Órdenes de trabajo, prioridad, asignación y seguimiento
Reportes	Reportes.fxml	ReportesController	Informes de ingresos y ocupación, exportación a PDF

4.3 Componentes Gráficos Utilizados

- TableView / TableColumn: Listados de clientes, reservas, empleados con columnas tipadas y ordenamiento.
- ComboBox: Selección de tipo de habitación, método de pago, estado de reserva.
- DatePicker: Selección de fechas de entrada y salida en formularios de reserva.
- TextField / PasswordField: Campos de texto con validación integrada.
- Button: Acciones CRUD con feedback visual (estados disable / enable).
- Label + Badge personalizado: BadgeUtil genera etiquetas de color según el estado de la entidad.
- Modales de confirmación: ModalUtil y ConfirmDialog para operaciones destructivas.
- Notificaciones Toast: NotificationUtil muestra mensajes de éxito, advertencia y error no intrusivos.
- Avatar de usuario: AvatarUtil muestra las iniciales del empleado activo en la barra de navegación.

4.4 Gestión de Eventos

Los eventos de la interfaz se gestionan mediante anotaciones @FXML en los métodos del controlador y el atributo onAction en el FXML. La navegación entre pantallas está centralizada en NavigatorUtil, un singleton que carga el FXML correspondiente y lo inyecta en el Stage principal.

// Ejemplo en ReservasController.java

@FXML

```
private void crearReserva(ActionEvent event) {
    try {
        ReservaDTO dto = construirDTO();
        ValidadorEntradas.validarFechas(dto.getFechaEntrada(), dto.getFechaSalida());
        ReservaService service = AppContext.getInstance().getReservaService();
        service.crearReserva(dto);
        NotificationUtil.mostrarExito("Reserva creada exitosamente");
        cargarTablaReservas();
    } catch (ExcepcionValidacion ex) {
        NotificationUtil.mostrarError(ex.getMessage());
    } catch (ExcepcionNegocio ex) {
        NotificationUtil.mostrarAdvertencia(ex.getMessage());
    }
}
```

4.5 Arquitectura en Tres Capas

El sistema aplica estrictamente la arquitectura de tres capas, donde cada capa depende únicamente de la inmediatamente inferior, nunca al revés.

Capa	Paquetes	Responsabilidad	Tecnología
Presentación	com.hotel.ui.controllers com.hotel.ui.components com.hotel.view	Mostrar datos al usuario, capturar eventos, validar entrada básica	JavaFX, FXML, CSS
Negocio (Servicio)	com.hotel.service com.hotel.service.strategy com.hotel.dto	Aplicar reglas de negocio, orquestar DAOs, calcular precios, enviar emails	Java puro, BCrypt, Jakarta Mail, OpenPDF
Datos (DAO)	com.hotel.dao.interfaces com.hotel.dao.impl com.hotel.util	Ejecutar SQL, mapear ResultSet a objetos, gestionar transacciones y pool de conexiones	JDBC, Oracle XE, PreparedStatement

El punto de integración de las capas es la clase `AppContext`, que actúa como Service Locator: en el arranque instancia todos los DAOs, los inyecta en los servicios, y expone los servicios a los controladores.

5. BUENAS PRÁCTICAS DE CÓDIGO

5.1 Reglas de Código Limpio (Clean Code)

Nombres Significativos

Todos los identificadores del proyecto siguen las convenciones Java y expresan claramente su propósito sin necesidad de comentarios adicionales:

- Clases (PascalCase): `ClienteDAOImpl`, `ReservaService`, `HabitacionNoDisponibleException`
- Métodos (camelCase, verbo + sustantivo): `calcularPrecioFinal()`, `verificarDisponibilidad()`, `generarFactura()`, `obtenerNombreCompleto()`
- Atributos (camelCase): `hashContrasena`, `fechaContratacion`, `precioBase`, `debeCambiarContrasena`
- Constantes (UPPER_SNAKE_CASE): `TASA_IVA`, `DESCUENTO_CLIENTE_VIP`, `MINUTOS_EXPIRACION_SESION`

Funciones con Responsabilidad Única (SRP)

Cada método realiza exactamente una cosa. Por ejemplo, en `FacturaService` los métodos `calcularImpuestos(double subtotal)` y `registrarPago(Factura factura, MetodoPago metodo)` están separados; el primero calcula y el segundo persiste.

Ausencia de Comentarios Innecesarios

No se usan comentarios que parafraseen el código. Solo se documentan con Javadoc las clases y métodos públicos del API, y únicamente cuando el comportamiento no es evidente por el nombre.

Formato Consistente

El proyecto sigue el estilo de codificación estándar de Java (Oracle Code Conventions): llaves en la misma línea, 4 espacios de indentación, máximo 120 caracteres por línea.

5.2 Comentarios Javadoc

Todas las clases del modelo, las interfaces DAO, los servicios y los DTOs incluyen comentarios Javadoc con descripción de la clase, autor y versión. Los métodos complejos documentan sus parámetros (@param), valor de retorno (@return) y excepciones lanzadas (@throws).

```
/**
 * Servicio de gestion de reservas. Orquesta la creacion, confirmacion
 * y cancelacion de reservas aplicando el flujo de estados definido
 * y la estrategia de descuentos vigente.
 *
 * @author Equipo Hotel System
 * @version 1.246
 */
public class ReservaService {

    /**
     * Crea una nueva reserva en estado PENDIENTE.
     *
     * @param dto datos de la reserva proporcionados desde la UI
     * @return la reserva persistida con su identificador asignado
     * @throws ExcepcionValidacion si las fechas son invalidas
     * @throws HabitacionNoDisponibleException si la habitacion no esta libre
     */
    public Reserva crearReserva(ReservaDTO dto) { ... }
}
```

5.3 Convenciones de Nomenclatura Java

Elemento	Convencion	Ejemplo en el proyecto
Clases / Interfaces	PascalCase	ClienteDAOImpl, IReservaDAO, TipoHabitacion
Métodos / Variables	camelCase	calcularPrecioFinal(), esVip, precioBase
Constantes	UPPER_SNAKE_CASE	TASA_IVA, MULTIPLICADOR_PRECIO, DESCUENTO_CLIENTE_VIP
Paquetes	minuscula con puntos	com.hotel.dao.impl, com.hotel.service.strategy
Enums	PascalCase + valores UPPER_SNAKE_CASE	EstadoHabitacion.FUERA_DE_SERVICIO

5.4 Validación de Datos y Manejo de Excepciones

La validación de entradas del usuario se centraliza en ValidadorEntradas (utilidad estática) y ValidacionCampo (componente de UI). La regla es: validar en la frontera del sistema (controladores y DTOs); en capas internas se confía en que los datos ya fueron validados.

Validacion	Método	Capa
Formato de email	ValidadorEntradas.validarEmail(String)	Presentacion / Servicio
Teléfono 10 dígitos	ValidadorEntradas.validarTelefono(String)	Presentacion
Documento no vacío	ValidadorEntradas.validarDocumento(String)	Presentacion
Fechas coherentes (entrada < salida)	ValidadorEntradas.validarFechas(LocalDate, LocalDate)	Servicio
Disponibilidad de habitación	ReservaService.verificarDisponibilidad()	Servicio / DAO
Contraseña BCrypt (costo 12)	GenerarHash.hashBCrypt(String)	Servicio

La estrategia de manejo de excepciones sigue el flujo:

1. La capa DAO captura SQLException → lanza ExcepcionBaseDatos (unchecked).
2. La capa de servicio captura excepciones de negocio → lanza ExcepcionNegocio o deja pasar ExcepcionBaseDatos.
3. El controlador captura todas las excepciones conocidas → muestra mensaje amigable mediante NotificationUtil.
4. El ManejadorExcepciones global captura cualquier excepción no prevista en hilos de JavaFX y la registra en consola sin romper la aplicación.

6. TABLA DE TRAZABILIDAD — PRINCIPIOS GRASP Y SOLID

6.1 Principios GRASP

Principio GRASP	Clase / Método concreto	Justificacion técnica
Information Expert (Experto en Información)	Reserva.calcularTotalDiasReserva() Habitacion.calcularPrecioFinal() Factura.estaPagada()	Cada clase calcula o consulta informacion que solo ella posee. Reserva tiene las fechas, por eso calcula los dias. Habitacion tiene el precio base y el tipo, por eso calcula el precio final. Factura conoce su estadoPago, por eso responde estaPagada().
Creator (Creador)	ReservaService.crearReserva(ReservaDTO) FacturaService.generarFactura(Reserva)	El servicio crea objetos de dominio porque contiene la informacion de inicializacion y la logica de negocio necesaria. ReservaService tiene el cliente, la habitacion y las fechas; es quien crea la Reserva.
Controllor (Controlador)	ReservasControllor.crearReserva(ActionEvent) LoginControllor.handleLogin(ActionEvent)	Los controladores JavaFX reciben los eventos de la UI y delegan el trabajo a los servicios. No contienen logica de

Principio GRASP	Clase / Método concreto	Justificación técnica
		negocio: su única responsabilidad es traducir eventos de usuario en llamadas de servicio.
Low Coupling (Bajo Acoplamiento)	Interfaces IReservaDAO, ICienteDAO, etc. AppContext como Service Locator	Los servicios dependen de las interfaces DAO, no de las implementaciones concretas. Si se cambia Oracle por PostgreSQL, solo cambia la implementación; los servicios no se modifican.
High Cohesion (Alta Cohesion)	AuthService (solo autenticación) PdfReporteService (solo PDF) EmailService (solo emails)	Cada servicio agrupa únicamente responsabilidades relacionadas. AuthService maneja tokens, BCrypt, permisos y sesiones — todo concerniente a autenticación — sin mezclar lógica de reservas o facturación.
Polymorphism (Polimorfismo)	TipoHabitacion.calcularPrecio(double) Descuento.calcularDescuento(double)	Habitacion.calcularPrecioFinal() invoca tipoHabitacion.calcularPrecio(precioBase) sin saber si es HabitacionSimple, HabitacionSuite u otra. ReservaService aplica descuento polimórficamente sin condicionales.
Pure Fabrication (Fabricación Pura)	ValidadorEntradas GenerarHash NavigatorUtil NotificationUtil	Clases de utilidad que no representan conceptos del dominio hotelero pero son necesarias para la cohesión y el bajo acoplamiento. Agrupan funcionalidades transversales.
Indirection (Indirección)	AppContext (Service Locator) BaseDAO.enTransaccion()	AppContext introduce una capa de indirección entre los controladores y los servicios concretos. BaseDAO.enTransaccion(callback) introduce indirección para que las implementaciones DAO no gestionen manualmente la transacción.
Protected Variations (Variaciones Protegidas)	Interfaz Descuento con tres implementaciones Interfaces ICienteDAO, IReservaDAO	Las interfaces protegen el código de las variaciones concretas. Añadir un nuevo tipo de descuento no requiere modificar ReservaService; se crea una nueva clase que implemente Descuento.

6.2 Principios SOLID

Principio SOLID	Clase / Método concreto	Justificación técnica
S — Single Responsibility (Responsabilidad Unica)	ClienteService: solo logica de negocio de clientes ClienteDAOImpl: solo acceso a BD EmailService: solo envio de emails PdfReporteService: solo PDF	Cada clase tiene una unica razon para cambiar. Si cambia el servidor SMTP solo se modifica EmailService; si cambia la logica de VIP solo ClienteService; si cambia el motor de PDF solo PdfReporteService.
O — Open/Closed (Abierto/Cerrado)	Jerarquia TipoHabitacion Jerarquia Descuento	Para anadir HabitacionPresidencial se crea una subclase nueva sin modificar TipoHabitacion ni Habitacion. Para un nuevo descuento basta crear una nueva implementacion de Descuento.
L — Liskov Substitution (Sustitucion de Liskov)	Cliente y Empleado como subtipos de Persona HabitacionSuite como subtipo de TipoHabitacion	Cualquier metodo que recibe una Persona funciona correctamente si recibe un Cliente o Empleado. Las subclases de TipoHabitacion no debilitan las postcondiciones de calcularPrecio() (siempre retornan un double positivo).
I — Interface Segregation (Segregacion de Interfaces)	IClienteDAO (operaciones CRUD) IClienteBusqueda (busquedas especializadas)	Las operaciones CRUD y las de busqueda avanzada se separaron en dos interfaces. Un componente que solo necesita buscar clientes depende de IClienteBusqueda sin conocer los metodos de insercion y eliminacion.
D — Dependency Inversion (Inversion de Dependencias)	ReservaService depende de IReservaDAO (interfaz), no de ReservaDAOImpl ClienteService depende de IClienteDAO e IClienteBusqueda	Los modulos de alto nivel (servicios) no dependen de modulos de bajo nivel (implementaciones concretas). Ambos dependen de abstracciones (interfaces). Permite inyectar mocks y cambiar la BD sin tocar los servicios.

7. GESTIÓN DEL PROYECTO CON GIT

7.1 Estructura de Repositorio

El proyecto está versionado con Git en el repositorio github.com/jdavidpulgarin/hotelnativo. El trabajo se organizó mediante ramas de función (feature branches) asignadas a cada integrante y se fusionó en la rama principal main al finalizar cada módulo.

7.2 Evidencia de Commits por Integrante

La siguiente tabla resume la evidencia de commits reales obtenida del repositorio GitHub. Para el detalle completo de los ~525 commits ver el documento GESTION GIT Reporte Final.docx.

Hash	Autor	Fecha	Mensaje del commit	Módulo afectado
cae08be	jdavidpulgarin	08/06/2026	fix: corregir errores tras merge de feature/desarrollador1-dao	main / integración
718316d	jdavidpulgarin	08/06/2026	Merge branch 'origin/feature/desarrollador2-ui'	main / integración
c21edbd	jdavidpulgarin	08/06/2026	Merge branch 'origin/feature/desarrollador1-dao'	main / integración
80e736d	jdavidpulgarin	08/06/2026	feat: agregar module-info.java con configuracion de modulos JavaFX	com.hotel (config)
543f91e	jdavidpulgarin	06/06/2026	feat: implementar validarSoloLetras y validarSoloNumeros	com.hotel.util
1e8402f	jdavidpulgarin	06/06/2026	feat: crear ManejadorExcepciones con uncaughtException global	com.hotel.util
b2ca225	jdavidpulgarin	06/06/2026	feat: crear interfaz Descuento con patron Strategy	com.hotel.service.strategy

Hash	Autor	Fecha	Mensaje del commit	Módulo afectado
f5ce117	jdavidpulgarin	06/06/2026	feat: implementar DescuentoVip con patron Strategy	com.hotel.service.strategy
45c102e	jdavidpulgarin	06/06/2026	feat: crear clase ConexionBaseDatos con constantes y constructor	com.hotel.util
a48b688	jdavidpulgarin	25/05/2026	feat: crear clase ClienteService con imports y documentacion	com.hotel.service
d8b2daf	jdavidpulgarin	25/05/2026	feat: crear clase EmailService con imports y documentacion Javadoc	com.hotel.service
fa5d0fd	jdavidpulgarin	24/05/2026	feat: crear clase ChatbotService con patrones NLP	com.hotel.service
ab94518	jdavidpulgarin	24/05/2026	feat: crear clase AuthService con constantes de roles	com.hotel.service
a4f75e8	jdavidpulgarin	23/05/2026	feat: inicializar proyecto hotelnativo	raiz del proyecto
7e45360	juandbuelvas	06/06/2026	feat: agregar Reservas.fxml para gestion de reservas	com.hotel.ui.fxml
5176f10	juandbuelvas	06/06/2026	feat: agregar Dashboard.fxml con sidebar y layout principal	com.hotel.ui.fxml
9f5eef8	juandbuelvas	06/06/2026	feat: agregar Login.fxml con diseno moderno y autenticacion	com.hotel.ui.fxml
4338613	juandabuelvas	24/05/2026	feat: agregar NotificationUtil para mensajes tipo toast	com.hotel.ui.components

Hash	Autor	Fecha	Mensaje del commit	Módulo afectado
e661eeb	juandbuelvas	24/05/2026	feat: agregar clase AvatarUtil con paleta de colores	com.hotel.ui.components
0c0380d	juandbuelvas	26/05/2026	feat: agregar EmpleadosController con estructura base	com.hotel.ui.controllers
ac5c197	juandbuelvas	26/05/2026	feat: agregar FacturacionController con configuracion de tabla	com.hotel.ui.controllers
001cabe	juandbuelvas	26/05/2026	feat: agregar HabitacionesController con carga asíncrona	com.hotel.ui.controllers
fdb350a	rcarlosbuelvas	05/06/2026	feat: agregar clase abstracta Persona con atributos comunes	com.hotel.model
3b6a191	rcarlosbuelvas	05/06/2026	feat: agregar clase Habitacion con enum, atributos y constructores	com.hotel.model
5b5baaf	rcarlosbuelvas	05/06/2026	feat: agregar clase Factura con enums, atributos y constructor	com.hotel.model
01aca62	rcarlosbuelvas	24/05/2026	feat: agregar clase BaseDAO con conexionBD y constructor	com.hotel.dao.impl
1f7f0db	rcarlosbuelvas	24/05/2026	feat: agregar clase AuditoriaDAOImpl con SQL insert	com.hotel.dao.impl
ecf6042	rcarlosbuelvas	24/05/2026	feat: agregar clase FacturaDAOImpl con constantes SQL	com.hotel.dao.impl
6643b55	rcarlosbuelvas	25/05/2026	feat: agregar excepcion ExcepcionNegocio para logica de negocio	com.hotel.exception

Hash	Autor	Fecha	Mensaje del commit	Módulo afectado
fbc7037	rcarlosbuelvas	25/05/2026	feat: agregar excepcion HabitacionNoDisponibleException	com.hotel.exception

7.3 Estado de Ramas

Rama	Última actividad	Detrás de main	Adelante de main	Estado
main	08/06/2026	— (rama base)	— (rama base)	Activa — rama principal por defecto
feature/lider-logica-negocio	08/06/2026	313 commits	0 commits	Todo integrado en main (jdavidpulgarin)
feature/desarrollador2-ui	06/06/2026	309 commits	0 commits	Todo integrado en main (juandbuelvas)
feature/desarrollador1-dao	05/06/2026	341 commits	0 commits	Todo integrado en main (rcarlosbuelvas)

7.4 Tabla de Tareas — Estado Actualizado

ID	Tarea	Responsable	Estado	Notas
T-01	Modelo de dominio (Persona, Cliente, Empleado, Habitacion, Reserva, Factura, CheckInOut, Mantenimiento)	R. Carlos Buelvas	Completada	Herencia, enums y encapsulamiento implementados
T-02	Definición de interfaces DAO (9 interfaces)	R. Carlos Buelvas	Completada	IClienteDAO, IReservaDAO, IHabitacionDAO, etc.
T-03	Implementación de DAOs con Oracle JDBC	R. Carlos Buelvas	Completada	ClienteDAOImpl, ReservaDAOImpl,

ID	Tarea	Responsable	Estado	Notas
				FacturaDAOImpl, etc.
T-04	Pool de conexiones Oracle (ConexionBaseDatos, BaseDAO)	Jarrison Pulgarin	Completada	BlockingQueue con 5 conexiones, Singleton
T-05	AuthService, ClienteService, HabitacionService, EmailService, ChatbotService	Jarrison Pulgarin	Completada	BCrypt, Strategy de descuentos, GeminiAPI
T-06	ValidadorEntradas (8 validadores), ManejadorExcepciones, GenerarHash	Jarrison Pulgarin	Completada	Utilidades del sistema implementadas
T-07	Diseño de 10 pantallas FXML	Juan D. Buelvas	Completada	Login, Dashboard, Clientes, Reservas, Habitaciones, CheckInOut, Facturacion, Empleados, Mantenimiento, Reportes
T-08	Controladores JavaFX (EmpleadosCtrl, FacturacionCtrl, HabitacionesCtrl, etc.)	Juan D. Buelvas	Completada	Con gestion de eventos @FXML
T-09	Componentes UI (AvatarUtil, BadgeUtil, NotificationUtil, ValidacionCampo)	Juan D. Buelvas	Completada	Componentes reutilizables en com.hotel.ui.components
T-10	Scripts SQL: tablas, procedimientos, triggers	Todos	Completada	PKG_ADMIN, PKG_HOTEL, trg_no_solapar, trg_vip
T-11	Manejo de excepciones personalizadas (5 clases)	R. Carlos Buelvas / Jarrison Pulgarin	Completada	ExcepcionBaseDatos, ExcepcionNegocio, etc.

ID	Tarea	Responsable	Estado	Notas
T-12	Integración de ramas en main (merges)	Jarrison Pulgarin	Completada	c21edbd, 718316d, cae08be
T-13	Pruebas integrales del flujo reserva → checkin → factura	Todos	En progreso	Scripts SQL ejecutados; pruebas de UI en curso
T-14	Presentación final y demostración del sistema	Todos	Pendiente	Fecha por definir con el docente

1. ARQUITECTURA DEL SISTEMA

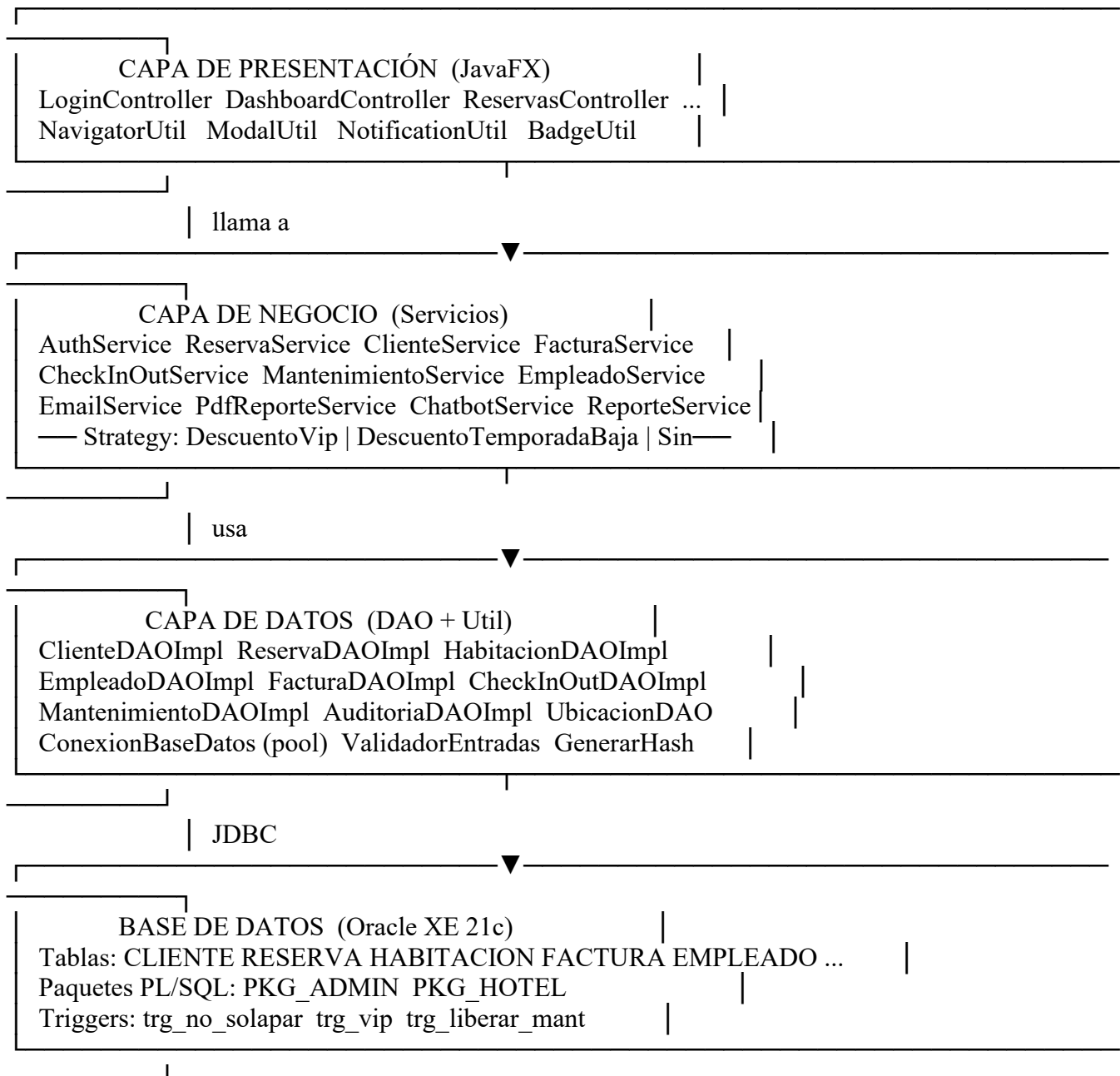
1.1 Descripción General

Hotel System es una aplicación de escritorio desarrollada en Java con JavaFX. Implementa una arquitectura de tres capas que separa claramente la presentación, la lógica de negocio y el acceso a datos. Esta separación garantiza bajo acoplamiento, alta cohesión y facilidad de mantenimiento.

1.2 Capas y su Responsabilidad

Capa	Paquete(s)	Responsabilidad	Tecnología
Presentación (UI)	com.hotel.ui.controllers com.hotel.ui.components com.hotel.view	Mostrar información al usuario, capturar eventos, validar entradas básicas y navegar entre pantallas.	JavaFX 11+, FXML, CSS
Negocio (Servicio)	com.hotel.service com.hotel.service.strategy com.hotel.dto	Aplicar reglas de negocio, orquestar DAOs, calcular precios, gestionar sesiones, enviar correos y generar PDFs.	Java puro, BCrypt, Jakarta Mail, OpenPDF, Gemini API
Datos (DAO)	com.hotel.dao.interfaces com.hotel.dao.impl com.hotel.util	Ejecutar consultas SQL, mapear ResultSet a objetos del dominio, gestionar el pool de conexiones y las transacciones.	JDBC, Oracle XE 21c, PreparedStatement, CallableStatement
Modelo (Dominio)	com.hotel.model	Representar las entidades del negocio (Cliente, Empleado, Reserva, Factura, etc.) con sus atributos, estados y comportamientos.	Java puro (POJOs, enums, clases abstractas)

1.3 Diagrama de Arquitectura (representación textual)



1.4 Flujo Principal de una Reserva

El flujo de creación de una reserva atraviesa las tres capas de la siguiente manera:

1. El usuario completa el formulario en ReservasController (Capa Presentación).
2. El controlador construye un ReservaDTO y llama a ReservaService.crearReserva(dto).
3. El servicio valida fechas, verifica disponibilidad y selecciona la estrategia de descuento.
4. ReservaService invoca ReservaDAOImpl.insertar(reserva), que ejecuta PKG_HOTEL.crear_reserva().
5. Oracle verifica el trigger trg_no_solapar antes de confirmar la inserción.

6. El servicio envía un correo de confirmación mediante EmailService.
7. El controlador actualiza la tabla de reservas y muestra una notificación de éxito.

2. DIAGRAMA DE PAQUETES FINAL

La estructura de paquetes del sistema refleja directamente la arquitectura en tres capas. Cada paquete agrupa clases con responsabilidad cohesionada y depende únicamente de los paquetes de capas inferiores.

com.hotel

- HotelApp.java ← Punto de entrada JavaFX
- AppContext.java ← Service Locator (Singleton)

model

- Persona.java (abstract)
- Cliente.java
- Empleado.java
- Habitacion.java
- TipoHabitacion.java (abstract)
- HabitacionSimple.java
- HabitacionDoble.java
- HabitacionMultiple.java
- HabitacionSuite.java
- Reserva.java
- Factura.java
- CheckInOut.java
- Mantenimiento.java
- Cargo.java
- Piso.java
- Auditoria.java

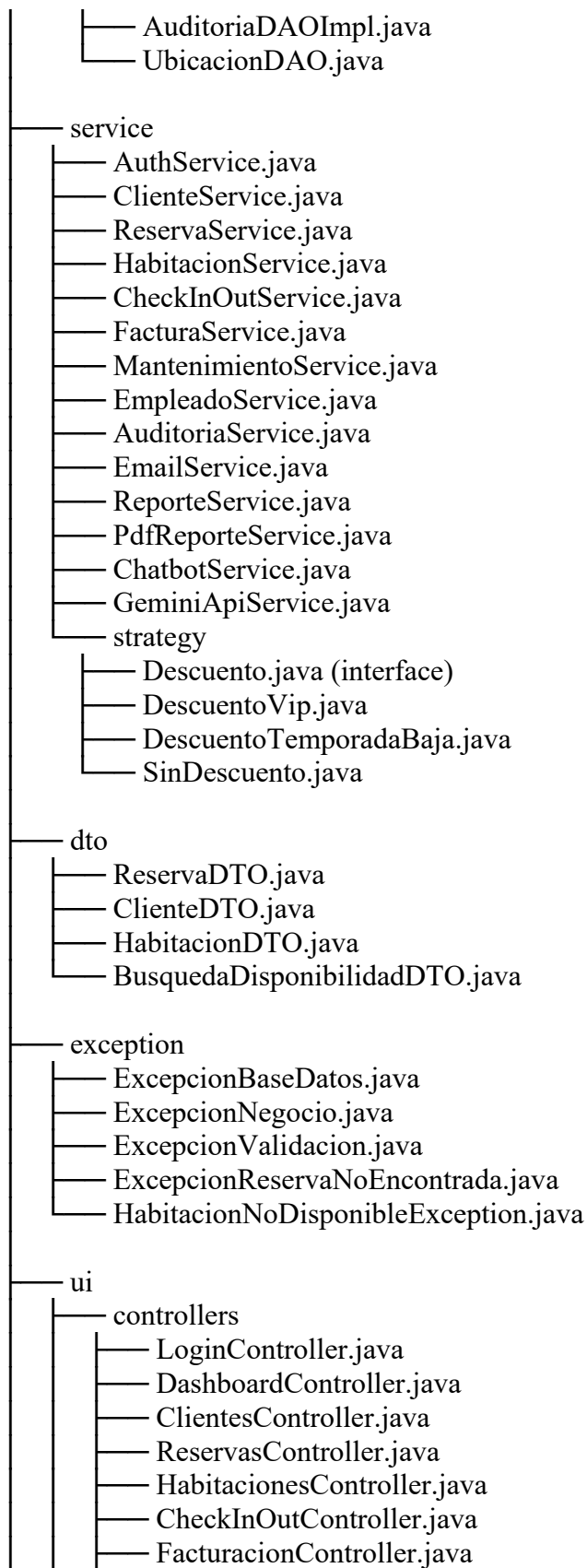
dao

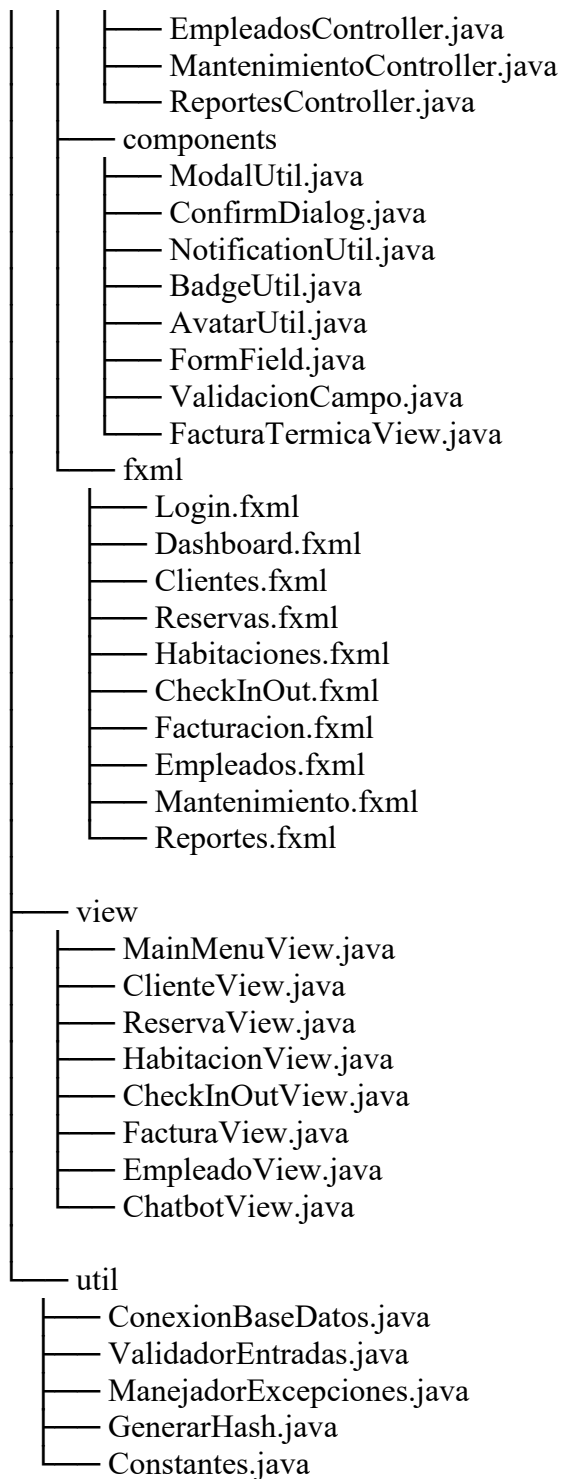
interfaces

- IClienteDAO.java
- IClienteBusqueda.java
- IReservaDAO.java
- IReservaBusqueda.java
- IHabitacionDAO.java
- IEmpleadoDAO.java
- IFacturaDAO.java
- ICheckInOutDAO.java
- IMantenimientoDAO.java
- IAuditoriaDAO.java

impl

- BaseDAO.java (abstract)
- ClienteDAOImpl.java
- ReservaDAOImpl.java
- HabitacionDAOImpl.java
- EmpleadoDAOImpl.java
- FacturaDAOImpl.java
- CheckInOutDAOImpl.java
- MantenimientoDAOImpl.java

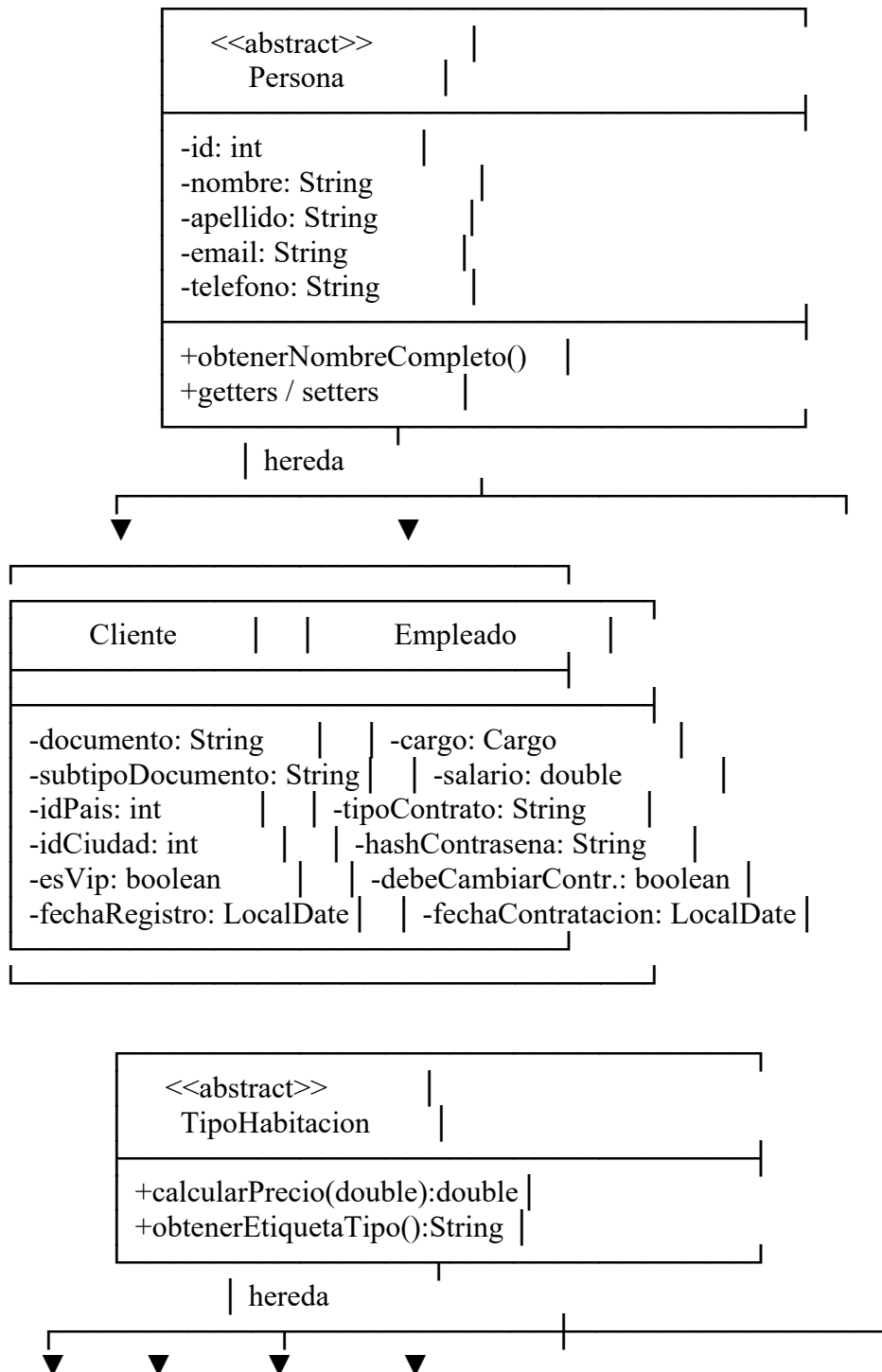


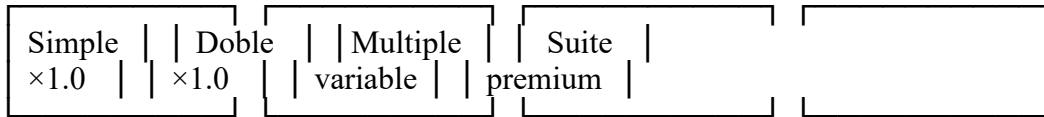


3. DIAGRAMA DE CLASES FINAL

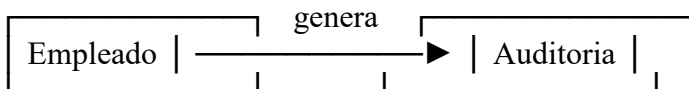
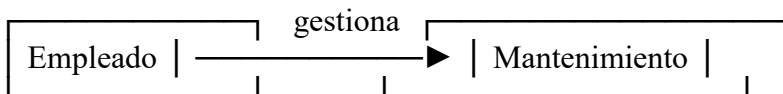
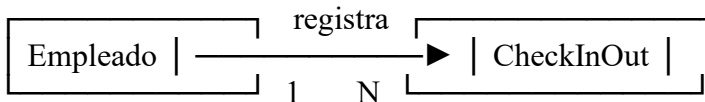
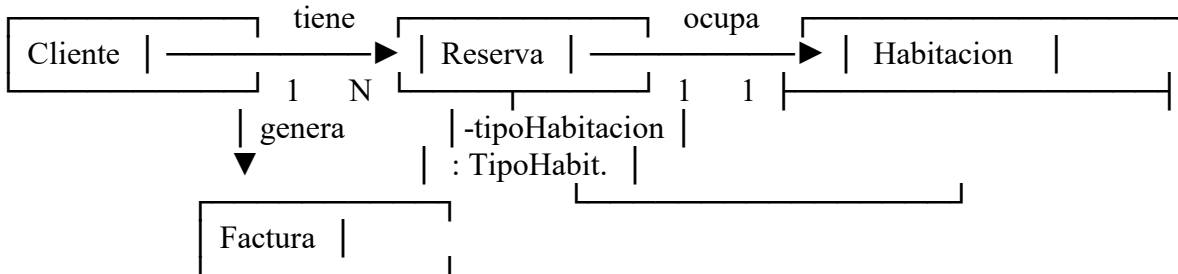
A continuación se presenta el diagrama de clases del modelo de dominio y las relaciones entre las principales entidades del sistema. Los símbolos usados son: «abstract» para clases abstractas, «interface» para interfaces, → para asociación, —▷ para herencia, ···▷ para realización de interfaz.

3.1 Jerarquía del Modelo de Dominio

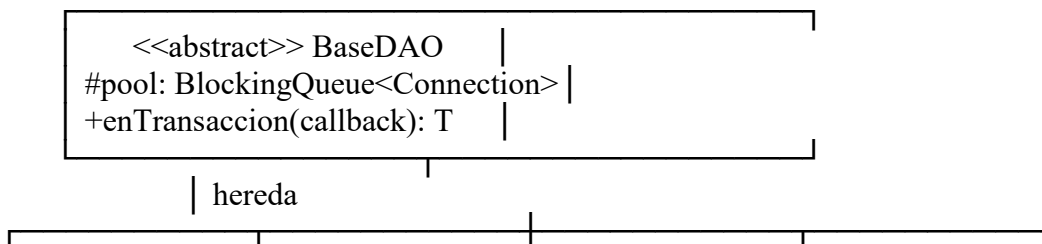




3.2 Relaciones entre Entidades del Dominio



3.3 Jerarquía DAO



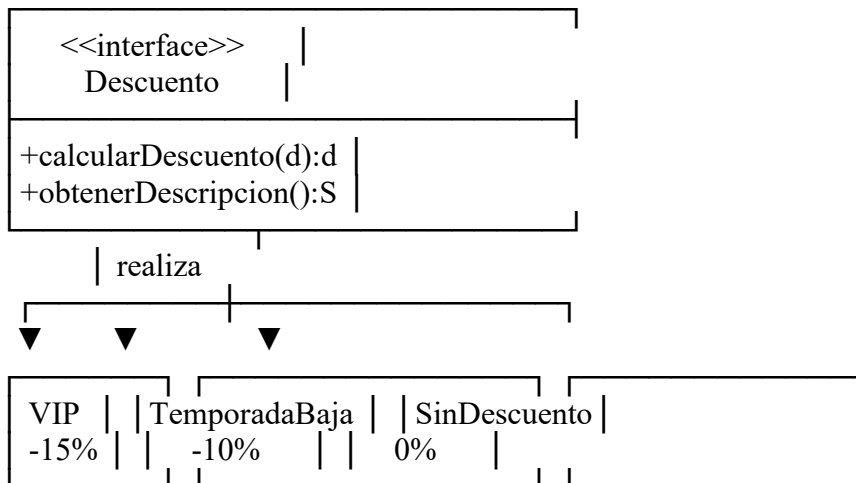
▼ ▼ ▼ ▼ ▼
 ClienteDAO ReservaDAO HabitDAO EmpleadoDAO ...
 Impl Impl Impl Impl

Cada Impl realiza (···▷) su(s) interfaz(ces) correspondiente(s):

ClienteDAOImpl ···▷ IClienteDAO, IClienteBusqueda
 ReservaDAOImpl ···▷ IReservaDAO, IReservaBusqueda
 HabitacionDAOImpl ···▷ IHabitacionDAO

...

3.4 Patrón Strategy — Descuentos



4. DESCRIPCIÓN DE CADA CLASE

4.1 Capa Modelo

Persona [abstract class]

Responsabilidad: Clase base de toda persona en el sistema. Centraliza los atributos comunes (identidad, contacto) para evitar duplicación en Cliente y Empleado.

Atributo	Tipo	Descripción
id	int	Clave primaria en base de datos
nombre	String	Primer nombre
apellido	String	Primer apellido
email	String	Correo electrónico (único en el sistema)
telefono	String	Número de teléfono de contacto

Método	Retorno	Descripción
obtenerNombreCompleto()	String	Concatena nombre y apellido para presentación en UI
getters / setters	varios	Encapsulamiento de todos los atributos privados

Cliente [class extends Persona]

Responsabilidad: Representa al huésped del hotel. Extiende Persona con datos de identificación, ubicación y el indicador VIP que activa descuentos automáticos del 15%.

Atributo	Tipo	Descripción
documento	String	Número de documento de identidad
subtipoDocumento	String	CC, CE, Pasaporte, NIT
segundoNombre	String	Segundo nombre (opcional)
apellido2	String	Segundo apellido
nacionalidad	String	País de nacionalidad
idPais	int	FK → tabla PAIS
idCiudad	int	FK → tabla CIUDAD
ciudadOrigen	String	Ciudad de procedencia del cliente
fechaRegistro	LocalDate	Fecha de primera interacción con el hotel
esVip	boolean	true si el cliente califica para descuento VIP

Método	Retorno	Descripción
isEsVip()	boolean	Retorna si el cliente es VIP

Método	Retorno	Descripción
toString()	String	Representación textual para depuración y logs
getters / setters	varios	Encapsulamiento completo

Empleado [class extends Persona]

Responsabilidad: Modela al trabajador del hotel con sus datos laborales y credenciales de autenticación. La contraseña nunca se almacena en texto plano; solo el hash BCrypt (costo 12).

Atributo	Tipo	Descripción
cargo	Cargo	Cargo asignado (FK → CARGO)
salario	double	Salario mensual bruto
tipoContrato	String	INDEFINIDO, FIJO o PRESTACIÓN
tipoPago	String	QUINCENAL o MENSUAL
fechaContratacion	LocalDate	Inicio del vínculo laboral
fechaFinContrato	LocalDate	Fin del contrato (null si indefinido)
hashContrasena	String	Hash BCrypt de la contraseña del empleado
debeCambiarContrasena	boolean	Fuerza cambio de contraseña en el próximo login

Método	Retorno	Descripción
getCargo()	Cargo	Retorna el cargo del empleado
getHashContrasena()	String	Retorna el hash (solo para AuthService)
isDebeCambiarContrasena()	boolean	Indica si se debe forzar cambio de clave

Habitacion [class]

Responsabilidad: Entidad central del sistema. Encapsula el estado de una habitación y expone métodos de transición que garantizan un ciclo de vida correcto y consistente.

Atributo	Tipo	Descripción
id	int	PK de la habitación
numero	String	Número de habitación (ej. "101")
tipoHabitacion	TipoHabitacion	Tipo polimórfico: Simple, Doble, Multiple, Suite
piso	int	Número de piso donde se ubica
estado	EstadoHabitacion	Estado actual (enum interno)

Atributo	Tipo	Descripción
precioBase	double	Precio base por noche sin descuentos
numCamas	int	Capacidad en número de camas

Método	Retorno	Descripción
estaDisponible()	boolean	true si estado == DISPONIBLE
reservar()	void	Transición estado → RESERVADA
ocupar()	void	Transición estado → OCUPADA
liberar()	void	Transición estado → DISPONIBLE
enviarAMantenimiento()	void	Transición estado → MANTENIMIENTO
calcularPrecioFinal()	double	Delega a tipoHabitacion.calcularPrecio(precioBase)

Reserva [class]

Responsabilidad: Gestiona el ciclo de vida completo de una reserva: desde PENDIENTE hasta COMPLETADA o CANCELADA. Cada método de transición garantiza que el flujo siga el orden correcto de estados.

Atributo	Tipo	Descripción
id	int	PK de la reserva
cliente	Cliente	Huésped que realiza la reserva
habitacion	Habitacion	Habitación reservada
fechaEntrada	LocalDate	Fecha de check-in
fechaSalida	LocalDate	Fecha de check-out
estado	EstadoReserva	PENDIENTE/CONFIRMADA/EN_PROCESO/COMPLETADA/CANCELADA
numPersonas	int	Número de huéspedes
precioTotal	double	Precio total calculado (con descuentos e IVA)
idCanal	int	Canal de venta: presencial, web, telefónico

Método	Retorno	Descripción
calcularTotalDiasReserva()	long	Días entre fechaEntrada y fechaSalida

Método	Retorno	Descripción
estaActiva()	boolean	true si estado no es CANCELADA ni COMPLETADA
confirmar()	void	PENDIENTE → CONFIRMADA
iniciarEstadia()	void	CONFIRMADA → EN_PROCESO
completar()	void	EN_PROCESO → COMPLETADA
cancelar()	void	Cualquier estado → CANCELADA

Factura [class]

Responsabilidad: Registra el comprobante de cobro de una reserva. Distingue el método de pago y almacena los campos específicos de cada modalidad (efectivo, tarjeta, transferencia).

Atributo	Tipo	Descripción
id	int	PK de la factura
reserva	Reserva	Reserva asociada
cliente	Cliente	Cliente facturado
fechaEmision	LocalDate	Fecha de generación
subtotal	double	Monto antes de impuestos
impuestos	double	IVA calculado (19% del subtotal)
total	double	subtotal + impuestos
estadoPago	EstadoPago	PENDIENTE, PAGADA o ANULADA
metodoPago	MetodoPago	EFFECTIVO, TARJETA_CREDITO, TARJETA_DEBITO, TRANSFERENCIA
montoRecibido	double	Efectivo recibido (solo pago en efectivo)
cambio	double	Vuelto al cliente (solo efectivo)
franquiciaTarjeta	String	Visa/Mastercard/Amex (solo tarjeta)
numCuotas	int	Número de cuotas (solo tarjeta crédito)
referenciaTransferencia	String	Código de transacción (solo transferencia)

Método	Retorno	Descripción
estaPagada()	boolean	true si estadoPago == PAGADA

CheckInOut [class]

Responsabilidad: Almacena la marca de tiempo de entrada (check-in) y salida (check-out) de cada huésped, vinculando la reserva con el empleado responsable del proceso.

Atributo	Tipo	Descripción
id	int	PK
reserva	Reserva	Reserva correspondiente
empleadoResponsable	Empleado	Empleado que registra la operación
fechaHoraCheckin	LocalDateTime	Timestamp de llegada del huésped
fechaHoraCheckout	LocalDateTime	Timestamp de salida (null si aún hospedado)
observaciones	String	Notas del empleado (opcional)

Método	Retorno	Descripción
haRealizadoCheckout()	boolean	true si fechaHoraCheckout no es null

Mantenimiento [class]

Responsabilidad: Representa una orden de trabajo de mantenimiento para una habitación. Registra el tipo, costo, estado y el empleado responsable de la intervención.

Atributo	Tipo	Descripción
id	int	PK
habitacion	Habitacion	Habitación intervenida
empleadoResponsable	Empleado	Técnico asignado
fechaSolicitud	LocalDate	Fecha de apertura de la orden
fechaRealizacion	LocalDate	Fecha de cierre (null si pendiente)
tipo	TipoMantenimiento	PREVENTIVO, CORRECTIVO o EMERGENCIA
estado	EstadoMantenimiento	SOLICITADO/EN_PROCESO/COMPLETADO/CANCELADO
costo	double	Costo total de la intervención
descripcionTrabajo	String	Detalle del trabajo realizado

Método	Retorno	Descripción
estaCompletado()	boolean	true si estado == COMPLETADO

Auditoria [class]

Responsabilidad: Registro inmutable de cada acción relevante ejecutada en el sistema. Permite trazabilidad completa: quién hizo qué, cuándo y sobre qué entidad.

Atributo	Tipo	Descripción
id	int	PK
fechaHora	LocalDateTime	Timestamp de la acción
idEmpleado	int	Empleado que ejecutó la acción
accion	String	Verbo de la acción: CREAR, ACTUALIZAR, CANCELAR, etc.
entidad	String	Nombre de la entidad afectada: RESERVA, CLIENTE, etc.
idEntidad	int	PK de la entidad afectada
detalles	String	Descripción adicional de la operación

Método	Retorno	Descripción
getters	varios	Solo getters (objeto inmutable — sin setters públicos)

4.2 Capa Servicio (principales)

Servicio	Dependencias inyectadas	Métodos clave	Responsabilidad
AuthService	EmpleadoDAO, AuditoriaService	autenticar(), generarToken(), verificarPermiso(), limpiarSesionesExpiradas()	Login, hashing BCrypt, tokens UUID de 8 horas, permisos por rol (ADMIN, RECEPCIONISTA, MANTENIMIENTO, CONTADOR)
ReservaService	ReservaDAO, HabitacionDAO,	crearReserva(), confirmarReserva(),	Orquesta el ciclo de vida de la reserva.

Servicio	Dependencias inyectadas	Métodos clave	Responsabilidad
	ClienteDAO, EmailService	cancelarReserva(), calcularPrecioTotal()	Selecciona la estrategia de descuento y envía notificaciones.
ClienteService	ClienteDAO, ClienteBusqueda, ReservaBusqueda	crearCliente(), buscarPorDocumento(), obtenerVips(), verificarDuplicados()	Gestión de clientes. Detecta duplicados por documento y email.
FacturaService	FacturaDAO, ReservaDAO	generarFactura(), calcularImpuestos(), registrarPago()	Genera facturas, aplica IVA 19% y registra el método de pago elegido.
CheckInOutService	CheckInOutDAO, ReservaDAO, EmpleadoDAO	realizarCheckin(), realizarCheckout(), validarCheckinTardy()	Controla la entrada y salida de huéspedes con marcas de tiempo.
EmailService	Jakarta Mail (SMTP)	enviarConfirmacion(), enviarRechazo(), enviarFactura()	Envía correos al cliente al confirmar o cancelar una reserva.
PdfReporteService	OpenPDF, FacturaDAO, ReservaDAO	generarPdfReporte(), generarFacturaPdf()	Exporta informes y facturas en formato PDF.
ChatbotService	GeminiApiService, todos los DAOs	procesarConsulta()	Chatbot impulsado por Google Gemini API para consultas de disponibilidad y reservas.

4.3 Capa DAO (principales)

Clase	Interfaces implementadas	Colección de retorno	Mecanismo SQL
BaseDAO (abstract)	—	BlockingQueue<Conne ction>	Pool de conexiones, método plantilla enTransaccion()
ClienteDAOImpl	IClienteDAO, IClienteBusqueda	ArrayList<Cliente>	PreparedStatement + LEFT JOIN PAIS/CIUDAD + paginación OFFSET/FETCH
ReservaDAOImpl	IReservaDAO, IReservaBusqueda	ArrayList<Reserva>	CallableStatement → PKG_HOTEL.crear_re serva(), cancelar()

Clase	Interfaces implementadas	Colección de retorno	Mecanismo SQL
HabitacionDAOImpl	IHabitacionDAO	ArrayList<Habitacion>	SELECT con mapeo de tipo → subclase concreta de TipoHabitacion
EmpleadoDAOImpl	IEmpleadoDAO	ArrayList<Empleado>	JOIN con CARGO, gestión de hash de contraseña
FacturaDAOImpl	IFacturaDAO	ArrayList<Factura>	INSERT/SELECT con cálculo de IVA y campos de pago
CheckInOutDAOImpl	ICheckInOutDAO	ArrayList<CheckInOut>	INSERT/UPDATE con timestamps LocalDateTime
MantenimientoDAOImpl	IMantenimientoDAO	ArrayList<Mantenimiento>	CRUD completo con filtros por estado y habitación
AuditoriaDAOImpl	IAuditoriaDAO	ArrayList<Auditoria>	Solo INSERT y SELECT por rango de fechas (solo lectura)

4.4 Clases de Utilidad

Clase	Patrón	Responsabilidad
ConexionBaseDatos	Singleton + Pool	Mantiene 5 conexiones Oracle activas en un BlockingQueue thread-safe. Inicialización lazy con double-checked locking.
AppContext	Singleton + Service Locator	Instancia y expone todos los DAOs y servicios al arrancar la aplicación. Punto central de inyección de dependencias.
ValidadorEntradas	Pure Fabrication (static)	Métodos estáticos de validación: email, teléfono, documento, fechas. Centraliza las reglas de validación del sistema.
GenerarHash	Pure Fabrication (static)	hashBCrypt(String) con costo 12 para contraseñas. hashSHA256() para otros usos.
Constantes	Clase de constantes	TASA_IVA=0.19, DESCUENTO_CLIENTE_VIP=0.15,

Clase	Patrón	Responsabilidad
		MINUTOS_EXPIRACION_SESSION=480, POOL_SIZE=5.
ManejadorExcepciones	Global Exception Handler	Thread.UncaughtExceptionHandler — captura excepciones no controladas en hilos JavaFX sin romper la aplicación.
NavigatorUtil	Singleton	Carga archivos FXML y cambia la escena del Stage principal. Punto único de navegación.

Relaciones principales identificadas:

Tabla origen	FK	Tabla referenciada	Cardinalidad	Descripción
CLIENTE	id_pais	PAIS	N : 1	País de origen del cliente
CLIENTE	id_ciudad	CIUDAD	N : 1	Ciudad de procedencia
RESERVA	id_cliente	CLIENTE	N : 1	Huésped que realiza la reserva
RESERVA	id_habitacion	HABITACION	N : 1	Habitación asignada
FACTURA	id_reserva	RESERVA	1 : 1	Factura generada al completar la reserva
CHECKIN_OUT	id_reserva	RESERVA	1 : 1	Registro de entrada/salida
CHECKIN_OUT	id_empleado	EMPLEADO	N : 1	Empleado que atendió
MANTENIMIENTO	id_habitacion	HABITACION	N : 1	Habitación intervenida
MANTENIMIENTO	id_empleado	EMPLEADO	N : 1	Técnico responsable
EMPLEADO	id_cargo	CARGO	N : 1	Cargo laboral del empleado
HABITACION	id_piso	PISO	N : 1	Piso del hotel
AUDITORIA	id_empleado	EMPLEADO	N : 1	Empleado que ejecutó la acción

6. DICCIONARIO DE DATOS

El diccionario de datos describe cada tabla de la base de datos Oracle con sus columnas, tipos de dato, restricciones y descripción del negocio.

Tabla: CLIENTE

Columna	Tipo Oracle	Restricción	Descripción
id_cliente	NUMBER	PK, NOT NULL	Identificador único del cliente (secuencia Oracle)
primer_nombre	VARCHAR2(100)	NOT NULL	Primer nombre del cliente
segundo_nombre	VARCHAR2(100)	NULL	Segundo nombre (opcional)
apellido_1	VARCHAR2(100)	NOT NULL	Primer apellido
apellido_2	VARCHAR2(100)	NULL	Segundo apellido
email	VARCHAR2(200)	NOT NULL, UNIQUE	Correo electrónico (clave de búsqueda)
telefono	VARCHAR2(20)	NULL	Teléfono de contacto
tipo_documento	VARCHAR2(20)	NOT NULL	CC / CE / PASAPORTE / NIT
numero_documento	VARCHAR2(50)	NOT NULL, UNIQUE	Número del documento de identidad
id_pais	NUMBER	FK → PAIS	País de origen del cliente
id_ciudad	NUMBER	FK → CIUDAD	Ciudad de procedencia
es_vip	CHAR(1)	DEFAULT "N"	S = cliente VIP, N = cliente regular
fecha_registro	DATE	DEFAULT SYSDATE	Fecha de primera reserva o registro manual

Tabla: EMPLEADO

Columna	Tipo Oracle	Restricción	Descripción
id_empleado	NUMBER	PK, NOT NULL	Identificador único del empleado
primer_nombre	VARCHAR2(100)	NOT NULL	Primer nombre
apellido_1	VARCHAR2(100)	NOT NULL	Primer apellido
email	VARCHAR2(200)	NOT NULL, UNIQUE	Correo corporativo (usado para login)
telefono	VARCHAR2(20)	NULL	Teléfono de contacto
id_cargo	NUMBER	FK → CARGO	Cargo del empleado en el hotel
salario	NUMBER(10,2)	NOT NULL	Salario mensual bruto
tipo_contrato	VARCHAR2(30)	NOT NULL	INDEFINIDO / FIJO / PRESTACION

Columna	Tipo Oracle	Restricción	Descripción
tipo_pago	VARCHAR2(20)	NOT NULL	QUINCENAL / MENSUAL
fecha_contratacion	DATE	NOT NULL	Fecha de inicio del contrato
fecha_fin_contrato	DATE	NULL	Fecha de fin (null si indefinido)
password_hash	VARCHAR2(255)	NOT NULL	Hash BCrypt costo 12 de la contraseña
debe_cambiar_password	CHAR(1)	DEFAULT "S"	S = debe cambiar en el próximo login

Tabla: HABITACION

Columna	Tipo Oracle	Restricción	Descripción
id_habitacion	NUMBER	PK, NOT NULL	Identificador único de la habitación
numero_habitacion	VARCHAR2(10)	NOT NULL, UNIQUE	Número visible (ej. "101", "205A")
tipo_habitacion	VARCHAR2(30)	NOT NULL	SIMPLE / DOBLE / MULTIPLE / SUITE
id_piso	NUMBER	FK → PISO	Piso en que se ubica la habitación
estado	VARCHAR2(30)	NOT NULL	DISPONIBLE/RESERVADA/OCUPADA/LIMPIEZA/MANTENIMIENTO/FUERA_DE_SERVICIO
precio_base	NUMBER(10,2)	NOT NULL	Precio por noche antes de descuentos
num_camras	NUMBER	NOT NULL	Número de camas disponibles

Tabla: RESERVA

Columna	Tipo Oracle	Restricción	Descripción
id_reserva	NUMBER	PK, NOT NULL	Identificador único de la reserva
id_cliente	NUMBER	FK → CLIENTE	Cliente que realiza la reserva
id_habitacion	NUMBER	FK → HABITACION	Habitación reservada
fecha_entrada	DATE	NOT NULL	Fecha de check-in
fecha_salida	DATE	NOT NULL	Fecha de check-out
estado	VARCHAR2(20)	NOT NULL	PENDIENTE/CONFIRMADA/EN_PROCESO

Columna	Tipo Oracle	Restricción	Descripción
			O/COMPLETADA/CANCELADA
num_personas	NUMBER	NOT NULL	Número de huéspedes incluidos
precio_total	NUMBER(10,2)	NOT NULL	Precio total (con descuentos e impuestos)
id_canal	NUMBER	NULL	Canal de venta: 1=presencial, 2=web, 3=telefónico

Tabla: FACTURA

Columna	Tipo Oracle	Restricción	Descripción
id_factura	NUMBER	PK, NOT NULL	Identificador único de la factura
id_reserva	NUMBER	FK → RESERVA	Reserva que origina la factura
subtotal	NUMBER(10,2)	NOT NULL	Monto antes de impuestos
impuestos	NUMBER(10,2)	NOT NULL	IVA = subtotal × 0.19
total	NUMBER(10,2)	NOT NULL	subtotal + impuestos
estado_pago	VARCHAR2(20)	NOT NULL	PENDIENTE / PAGADA / ANULADA
metodo_pago	VARCHAR2(30)	NULL	EFFECTIVO/TARJETA_CREDITO/TARJETA_DEBITO/TRANSFERENCIA
fecha_emision	DATE	DEFAULT SYSDATE	Fecha de generación de la factura
monto_recibido	NUMBER(10,2)	NULL	Monto en efectivo recibido del cliente
cambio	NUMBER(10,2)	NULL	Vuelto entregado al cliente
franquicia_tarjeta	VARCHAR2(30)	NULL	Visa, Mastercard, Amex, etc.
num_cuotas	NUMBER	NULL	Número de cuotas (tarjeta de crédito)
referencia_transferencia	VARCHAR2(100)	NULL	Código de transacción bancaria

Tabla: CHECKIN_OUT

Columna	Tipo Oracle	Restricción	Descripción
id_checkin_out	NUMBER	PK, NOT NULL	Identificador del registro
id_reserva	NUMBER	FK → RESERVA	Reserva asociada al check-in/out
id_empleado	NUMBER	FK → EMPLEADO	Empleado que atendió al huésped
fecha_hora_checkin	TIMESTAMP	NOT NULL	Fecha y hora exacta de llegada
fecha_hora_checkout	TIMESTAMP	NULL	Fecha y hora de salida (null si aún hospedado)
observaciones	VARCHAR2(500)	NULL	Notas adicionales del empleado

Tabla: MANTENIMIENTO

Columna	Tipo Oracle	Restricción	Descripción
id_mantenimiento	NUMBER	PK, NOT NULL	Identificador de la orden de trabajo
id_habitacion	NUMBER	FK → HABITACION	Habitación en intervención
id_empleado	NUMBER	FK → EMPLEADO	Técnico responsable de la tarea
tipo	VARCHAR2(20)	NOT NULL	PREVENTIVO / CORRECTIVO / EMERGENCIA
estado	VARCHAR2(20)	NOT NULL	SOLICITADO / EN_PROCESO / COMPLETADO / CANCELADO
fecha_solicitud	DATE	DEFAULT SYSDATE	Fecha de apertura de la orden
fecha_realizacion	DATE	NULL	Fecha de cierre de la orden
costo	NUMBER(10,2)	NULL	Costo total de la intervención
descripcion_trabajo	VARCHAR2(1000)	NULL	Detalle del trabajo realizado

Tabla: AUDITORIA

Columna	Tipo Oracle	Restricción	Descripción
id_auditoria	NUMBER	PK, NOT NULL	Identificador del registro de auditoría

Columna	Tipo Oracle	Restricción	Descripción
fecha_hora	TIMESTAMP	NOT NULL	Momento exacto de la acción
id_empleado	NUMBER	FK → EMPLEADO	Empleado que ejecutó la acción
accion	VARCHAR2(50)	NOT NULL	CREAR / ACTUALIZAR / ELIMINAR / CANCELAR / LOGIN / LOGOUT
entidad	VARCHAR2(50)	NOT NULL	Nombre de la tabla/entidad afectada
id_entidad	NUMBER	NOT NULL	PK de la fila afectada en la entidad
detalles	VARCHAR2(1000)	NULL	Descripción adicional de la operación ejecutada

Tabla: CARGO

Columna	Tipo Oracle	Restricción	Descripción
id_cargo	NUMBER	PK, NOT NULL	Identificador único del cargo
nombre_cargo	VARCHAR2(100)	NOT NULL, UNIQUE	Nombre del cargo (ADMIN, RECEPCIONISTA, etc.)
descripcion	VARCHAR2(500)	NULL	Descripción de las responsabilidades del cargo
salario_base	NUMBER(10,2)	NOT NULL	Salario base sugerido para el cargo

Tabla: PISO

Columna	Tipo Oracle	Restricción	Descripción
id_piso	NUMBER	PK, NOT NULL	Identificador del piso
numero_piso	NUMBER	NOT NULL	Número de piso (1, 2, 3...)
descripcion	VARCHAR2(200)	NULL	Descripción o nombre del piso (ej. "Piso Ejecutivo")
cantidad_habitaciones	NUMBER	NOT NULL	Número de habitaciones en ese piso

7. EVIDENCIA DE PRINCIPIOS APLICADOS

La siguiente tabla consolida los patrones GRASP, los principios SOLID y las prácticas de Código Limpio presentes en la solución final, con referencia directa a la clase o método donde se evidencia cada uno.

7.1 Principios GRASP

Principio GRASP	Clase / Método	Justificación técnica
Information Expert (Experto en Información)	Reserva.calcularTotalDiasReserva() Habitacion.calcularPrecioFinal() Factura.estaPagada()	Cada clase calcula lo que solo ella sabe: Reserva tiene las fechas → calcula días. Habitacion tiene precio base y tipo → calcula precio final. Factura conoce estadoPago → responde estaPagada().
Creator (Creador)	ReservaService.crearReserva(ReservaDTO) FacturaService.generarFactura(Reserva)	El servicio crea objetos de dominio porque posee toda la información de inicialización y las reglas de negocio necesarias para construirlos correctamente.
Controller (Controlador)	ReservasController.crearReserva(ActionEvent) LoginController.handleLogin(ActionEvent)	Los controladores JavaFX reciben eventos de la UI y los delegan a los servicios. No contienen lógica de negocio: traducen eventos en llamadas de servicio.
Low Coupling (Bajo Acoplamiento)	IReservaDAO, IClienteDAO y demás interfaces DAO AppContext como Service Locator	Los servicios dependen de interfaces, no de implementaciones concretas. Cambiar Oracle por otro motor solo afecta las clases impl.*
High Cohesion (Alta Cohesión)	AuthService (solo autenticación) EmailService (solo correos) PdfReporteService (solo PDF)	Cada servicio agrupa únicamente responsabilidades relacionadas entre sí. AuthService gestiona tokens, BCrypt y permisos sin mezclar lógica de reservas.
Polymorphism (Polimorfismo)	TipoHabitacion.calcularPrecio(double) Descuento.calcularDescuento(double)	Habitacion.calcularPrecioFinal() invoca polimórficamente sin conocer el tipo concreto. ReservaService aplica descuento sin condicionales if/else sobre el tipo.

Principio GRASP	Clase / Método	Justificación técnica
Pure Fabrication (Fabricación Pura)	ValidadorEntradas, GenerarHash NavigatorUtil, NotificationUtil	Clases de utilidad sin representación en el dominio hotelero. Agrupan funcionalidades transversales para mejorar cohesión y reducir duplicación.
Indirection (Indirección)	AppContext (Service Locator) BaseDAO.enTransaccion(TransaccionCallback)	AppContext desacopla controladores de implementaciones concretas. enTransaccion() desacopla el manejo de transacciones de cada método DAO.
Protected Variations (Variaciones Protegidas)	Interfaz Descuento + DescuentoVip, DescuentoTemporadaBaja Interfaces IClienteDAO, IReservaDAO	Las interfaces protegen el código de las variaciones internas. Añadir DescuentoCorporativo solo requiere crear una nueva clase que implemente Descuento.

7.2 Principios SOLID

Principio SOLID	Clase / Método	Justificación técnica
S — Single Responsibility (Responsabilidad Única)	ClienteService (negocio de clientes) ClienteDAOImpl (acceso a BD) EmailService (correos) PdfReporteService (PDF)	Cada clase tiene una única razón para cambiar. Si cambia el servidor SMTP, solo se modifica EmailService. Si cambia la lógica VIP, solo ClienteService.
O — Open/Closed (Abierto/Cerrado)	Jerarquía TipoHabitacion Jerarquía Descuento (interface)	Para añadir HabitacionPresidencial se crea una subclase nueva sin modificar TipoHabitacion ni Habitacion. Para un nuevo descuento basta una nueva implementación.
L — Liskov Substitution (Sustitución de Liskov)	Cliente y Empleado como subtipos de Persona HabitacionSuite subtipo de TipoHabitacion	Todo lugar que espera Persona funciona con Cliente o Empleado sin cambios. Las subclases de TipoHabitacion no debilitan postcondiciones (siempre retornan $\text{double} \geq 0$).
I — Interface Segregation (Segregación de Interfaces)	IClienteDAO (CRUD) IClienteBusqueda (búsquedas especializadas)	Las operaciones CRUD y búsqueda avanzada se separaron. Un componente que

Principio SOLID	Clase / Método	Justificación técnica
		solo busca clientes depende de IClienteBusqueda sin conocer insertar/eliminar.
D — Dependency Inversion (Inversión de Dependencias)	ReservaService depende de IReservaDAO ClienteService depende de IClienteDAO	Módulos de alto nivel (servicios) no dependen de implementaciones concretas (DAOImpl). Ambos dependen de abstracciones (interfaces). Permite cambiar Oracle sin modificar servicios.

7.3 Prácticas de Código Limpio (Clean Code)

Práctica Clean Code	Evidencia en el código	Clase / Ejemplo
Nombres significativos	Todos los identificadores expresan su propósito sin comentarios adicionales.	calcularPrecioFinal(), verificarDisponibilidad(), debeCambiarContrasena, TASA_IVA
Funciones con responsabilidad única	Cada método hace exactamente una cosa. Si necesita más, delega a métodos auxiliares.	FacturaService: calcularImpuestos() y registrarPago() separados
Ausencia de comentarios innecesarios	No se comentan líneas que el código ya explica por sí solo. Solo Javadoc en API pública.	Todos los servicios y DAOs; solo Javadoc en clases e interfaces públicas
Uso de enums en lugar de magic strings	Los estados se representan con enums tipados, evitando comparaciones de cadenas frágiles.	EstadoHabitacion.DISPONIBLE, MetodoPago.EFECTIVO, EstadoReserva.CONFIRMADA
Excepciones específicas (no Exception genérico)	Se lanzan excepciones con semántica clara según la capa y el tipo de error.	ExcepcionBaseDatos, HabitacionNoDisponibleException, ExcepcionValidacion
Validación en la frontera del sistema	Las entradas se validan en controladores/servicios. Las capas internas confían en que los datos ya fueron validados.	ValidadorEntradas.validarEmail(), ValidacionCampo en componentes UI
Convenciones Java (PascalCase / camelCase / UPPER_SNAKE_CASE)	Se respetan en el 100% del código fuente.	ClienteDAOImpl (clase), calcularDescuento() (método), DESCUENTO_CLIENTE_VIP (constante)
No repetición de código (DRY)	La lógica de pool y transacciones está en BaseDAO y es heredada por todos los DAOs.	BaseDAO.enTransaccion() usado por los 9 DAOs implementados

Práctica Clean Code	Evidencia en el código	Clase / Ejemplo
Optional para evitar NullPointerException	Los métodos buscarPorId() retornan Optional<T>, obligando al llamador a manejar la ausencia.	IClienteDAO.buscarPorId(int): Optional<Cliente>
Colecciones thread-safe donde se requiere concurrencia	Sesiones y credenciales usan ConcurrentHashMap para acceso concurrente sin locks explícitos.	AuthService.sesionesActivas: ConcurrentHashMap<String, SesionInfo>

CONCLUSIÓN GENERAL

Logros Alcanzados

El desarrollo del proyecto Hotel Nativo permitió construir una aplicación de gestión hotelera completamente funcional que integra, en una sola plataforma, los diez módulos operativos del negocio: clientes, reservas, habitaciones, check-in/out, facturación, empleados, mantenimiento, reportes, auditoría y un chatbot inteligente. Este resultado demuestra que el equipo logró superar el umbral de un ejercicio académico para construir un sistema con características de software empresarial real.

Desde el punto de vista técnico, se implementaron con éxito todos los elementos exigidos por el programa académico: la jerarquía de clases con herencia y polimorfismo, el patrón DAO para el acceso a datos, la arquitectura en tres capas, las interfaces gráficas con JavaFX, y la aplicación documentada de los principios GRASP y SOLID.

Aplicación de Conceptos de Programación Orientada a Objetos

El proyecto evidencia la aplicación práctica de los cuatro pilares de la programación orientada a objetos en un contexto real:

- **Abstracción:** Las clases abstractas *Persona* y *TipoHabitacion* definen contratos que ocultan los detalles de implementación. Los servicios trabajan con interfaces DAO sin conocer cómo se ejecuta el SQL internamente.
- **Encapsulamiento:** Todos los atributos del modelo son *private* y se exponen únicamente a través de getters y setters. La contraseña del empleado nunca sale del sistema en texto plano; siempre se maneja el hash BCrypt.
- **Herencia:** Las jerarquías *Cliente/Empleado* (extienden *Persona*) y *HabitacionSimple/HabitacionDoble/HabitacionMultiple/HabitacionSuite* (extienden *TipoHabitacion*) eliminan la duplicación de código y establecen relaciones semánticas claras entre las entidades del dominio.
- **Polimorfismo:** *Habitacion.calcularPrecioFinal()* invoca *tipoHabitacion.calcularPrecio()* sin conocer el tipo concreto. *ReservaService* aplica el descuento correcto (VIP, temporada baja o ninguno) mediante la interfaz *Descuento* sin condicionales *if/else*, gracias al patrón *Strategy*.

Principios de Diseño Aplicados

La tabla de trazabilidad elaborada en las fases 3 y 4 demuestra que los nueve patrones GRASP y los cinco principios SOLID no fueron aplicados de forma superficial, sino que guiaron decisiones de diseño concretas con impacto medible en la calidad del código:

Principio	Impacto concreto en el sistema
Information Expert (GRASP)	Cada clase es responsable de calcular lo que solo ella sabe: Reserva calcula sus días, Habitacion su precio final, Factura si está pagada.
Low Coupling (GRASP)	Los servicios dependen de interfaces DAO, no de implementaciones Oracle. Cambiar la base de datos no requiere modificar ningún servicio.

High Cohesion (GRASP)	AuthService maneja solo autenticacion. EmailService solo correos. PdfReporteService solo PDF. Cada clase tiene una responsabilidad delimitada.
Single Responsibility (SOLID-S)	ClienteService, ClienteDAOImpl y ClientesController gestionan al cliente desde tres perspectivas distintas, cada una con una sola razón para cambiar.
Open/Closed (SOLID-O)	Agregar HabitacionPresidencial o DescuentoCorporativo solo requiere crear nuevas clases, sin modificar el código existente y probado.
Dependency Inversion (SOLID-D)	ReservaService depende de IReservaDAO (abstracción), no de ReservaDAOImpl (implementacion concreta). Facilita pruebas con mocks y cambios tecnológicos.

Trabajo Colaborativo y Gestión con Git

El repositorio github.com/jdavidpulgarin/hotelnativo acumula aproximadamente 525 commits distribuidos entre los tres integrantes del equipo a lo largo de 17 días de desarrollo activo (del 23 de mayo al 8 de junio de 2026). La estrategia de ramas de función (feature branches) permitió que los tres módulos principales —modelo/DAO, interfaz gráfica y lógica de negocio— se desarrollaran en paralelo sin interferencias, y se integraron exitosamente en la rama main mediante merges controlados.

La adopción de la convención Conventional Commits (feat:, fix:) hizo que el historial del repositorio sea legible y auditable, lo que refleja madurez en el manejo de herramientas profesionales de control de versiones. Cada integrante tiene una participación verificable y proporcional a su módulo asignado:

Integrante	Commits aprox.	Porcentaje	Ramas de función
Jarrison Pulgarin (jdavidpulgarin)	~ 313	~31%	feature/lider-logica-negocio + main
Juan D. Buelvas (juandbuelvas)	~ 309	~30%	feature/desarrollador2-ui
R. Carlos Buelvas (rcarlosbuelvas)	~ 341	~39%	feature/desarrollador1-dao
TOTAL	~953	100%	4 ramas activas

Lecciones Aprendidas

El desarrollo del sistema dejó aprendizajes fundamentales que trascienden el código:

- La arquitectura decide antes de codificar. Definir la separación en tres capas desde el inicio evitó el "código espagueti" y permitió que cada integrante trabajara en su módulo sin bloquear al resto del equipo.
- Las interfaces son contratos de equipo. Al definir las interfaces DAO en la fase de diseño, los desarrolladores de servicios (Jarrison) y de implementaciones (Carlos) pudieron trabajar en paralelo usando los contratos acordados.
- El control de versiones es comunicación. Cada commit con un mensaje descriptivo elimina la necesidad de reuniones de sincronización frecuentes y crea una bitácora permanente de las decisiones de implementación.

- La seguridad no es opcional. La decisión de usar BCrypt con costo 12, excluir las credenciales del repositorio (.gitignore) y gestionar sesiones con tokens UUID de expiración automática refleja que la seguridad debe considerarse desde el diseño, no añadirse al final.
- Los patrones de diseño reducen la complejidad. El patrón Strategy evitó bloques if/else para descuentos; el patrón DAO desacopló la BD del negocio; el Singleton garantizó un único pool de conexiones thread-safe. Cada patrón resolvió un problema concreto, no fue aplicado por moda.

Trabajo Futuro y Posibles Mejoras

El sistema construido constituye una base sólida sobre la cual se pueden incorporar mejoras significativas en futuras versiones:

Área de mejora	Descripción
Pruebas automatizadas	Implementar pruebas unitarias con JUnit 5 y pruebas de integración con una base de datos de prueba embebida (H2) para verificar el comportamiento de los DAOs y servicios de forma automatizada.
Módulo de reservas en línea	Exponer una API REST (con Spring Boot o Quarkus) que permita a los huéspedes realizar reservas desde un portal web o aplicación móvil, integrando el motor de disponibilidad ya implementado.
Dashboard analítico avanzado	Incorporar gráficos de tendencias de ocupación, análisis de ingresos por temporada y proyecciones de demanda utilizando librerías como JFreeChart o conectando con herramientas de Business Intelligence.
Integración con pasarela de pagos	Conectar el módulo de facturación con pasarelas de pago reales (PayU, Wompi, Stripe) para procesar transacciones en línea de forma segura.
Internacionalización (i18n)	Implementar soporte multiidioma (español, inglés) para atender huéspedes internacionales, usando ResourceBundle de Java.
Migración a base de datos en la nube	Migrar de Oracle XE local a Oracle Autonomous Database o PostgreSQL en la nube para mejorar la disponibilidad y eliminar la dependencia de infraestructura local.

Reflexión Final

El proyecto Hotel System representa mucho más que el cumplimiento de un requisito académico. Es la demostración de que un equipo de tres estudiantes de cuarto semestre, aplicando metodologías y principios de ingeniería de software bien fundamentados, es capaz de construir un sistema de información complejo, funcional y mantenible en un plazo de cuatro semanas.

Cada decisión técnica documentada en las cuatro fases del proyecto —desde la elección de la arquitectura en tres capas hasta la aplicación del patrón Strategy para los descuentos— refleja un proceso

de pensamiento orientado a la calidad del software, no simplemente a hacer que el código funcione. Esta es, quizás, la lección más valiosa del proyecto: escribir software profesional no consiste solo en lograr que el programa ejecute sin errores, sino en construirlo de tal manera que otros desarrolladores puedan entenderlo, extenderlo y mantenerlo con confianza.

La experiencia de trabajar con Git en un repositorio compartido, resolver conflictos de integración, revisar el código de los compañeros y documentar técnicamente el sistema construido son competencias transversales que el equipo lleva consigo como preparación real para el ejercicio profesional de la ingeniería de sistemas.